Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки (специальность) 10.05.03 «Информационная безопасность автоматизированных систем»
(код и наименование направления подготовки)
Профиль/программа/направленность Обеспечение информационной безопасности распределенных информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
Дипломная работа
(вид выпускной квалификационной работы – бакалаврская работа, дипломная работа (проект), магистерская диссертация)

Кульшина Дмитрия Вячеславовича
(фамилия, имя, отчество обучающегося)

Тема: Сценарии реализации и меры противодействия атакам на средства машинного обучения: алгоритмическое и программное обеспечение

Состав выпускной квалификационной работы:
Расчетно-пояснительная записка на 100 страницах
Графическая часть на _____ листах

Расчетно-пояснительная записка к выпускной квалификационной работе:
Заведующий кафедрой _____ (А.Г. Остапенко)
(подпись) (инициалы, фамилия)

Руководитель _____ (А.С. Пахомова)
(подпись) (инициалы, фамилия)

Консультанты:
по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

Обучающийся _____ Д.В. Кульшин
(подпись) (инициалы, фамилия)

«02» 07 2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности

Кафедра систем информационной безопасности

Направление подготовки/специальность 10.05.03 «Информационная безопасность автоматизированных систем»
(код, наименование)

Профиль/программа/направленность Обеспечение информационной безопасности распределенных информационных систем

Утверждаю _____
(дата)

Зав.кафедрой _____
(подпись)

ЗАДАНИЕ

ПО ПОДГОТОВКЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

дипломная работа
дипломная работа (проект), бакалаврская работа, магистерская диссертация

Обучающемуся Кульшину Дмитрию Вячеславовичу
фамилия, имя, отчество

1. Тема: Сценарии реализации и меры противодействия атакам на средства машинного обучения: алгоритмическое и программное обеспечение

ФГБОУ ВО «ВГТУ», ВГТУ
по заданию какой организации выполняется работа

приказ № 01-1-10/3-2979 от 25.06.2025
утверждена приказом по университету

2. Срок сдачи обучающимся выпускной квалификационной работы 02.07.25

3. Исходные данные —

4. Краткое содержание выпускной квалификационной работы

Глава 1. Анализ средств автоматизации описания атак на средства машинного обучения и оценки рисков их реализации
перечень подлежащих разработке в выпускной квалификационной работе вопросов

Глава 2. Разработка модулей системы по оценке рисков и генерации мер противодействия для атак на средства машинного обучения

Глава 3. Управление информационной системой

5. Перечень графического материала (с точным указанием обязательных чертежей по разделам

6. Консультанты (с указанием относящихся к ним разделов)

График выполнения выпускной квалификационной работы

№ п/п	Наименование разделов выпускной квалификационной работы	Срок выполнения этапов работы
1	Введение	22.03.25 – 22.05.25
2	Анализ средств автоматизации описания атак на средства машинного обучения и оценки рисков их реализации	22.03.25 – 22.05.25
3	Разработка модулей системы по оценке рисков и генерации мер противодействия для атак на средства машинного обучения	23.05.25 – 31.05.25
4	Управление информационной системой	1.06.25 – 24.06.25
5	Заключение	25.06.25 – 02.07.25

Руководитель _____ (подпись)

Дата выдачи задания 22.03.25

Задание принял к исполнению 22.03.25

Подпись обучающегося _____ (дата)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»

АННОТАЦИЯ
выпускной квалификационной работы

Обучающегося Кульшина Дмитрия Вячеславовича

Тема ВКР: Сценарии реализации и меры противодействия атакам на средства машинного обучения: алгоритмическое и программное обеспечение

Характеристика выпускной квалификационной работы

Актуальность исследования

В условиях стремительного расширения областей применения средств машинного обучения – от безопасности до медицины и финансов – растет и число атак, эксплуатирующих их уязвимости. В то же время отсутствуют автоматизированные методы представления сведений об атаках на алгоритмы машинного обучения, а также комплексные алгоритмические и программные решения для сбора информации об уязвимостях, оценки рисков и выработки мер противодействия. Это порождает противоречие между высокой востребованностью средств машинного обучения и недостаточной защищенностью информационных автоматизированных систем, использующих их.

Задачи, решаемые в ВКР

– анализ существующих информационных ресурсов, содержащих сведения об атаках на средства машинного обучения на предмет наличия недостатков функциональных особенностей;

– разработка алгоритма функционирования и взаимодействия модулей, обеспечивающих агрегацию данных об уязвимостях, оценку рисков и выработку мер противодействия для атак, направленных на средства машинного обучения;

– реализация программных модулей, обеспечивающих агрегирование данных об уязвимостях, оценку риска и выработку мер противодействия для атак, направленных на средства машинного обучения.

Структура ВКР

Раздел 1. Анализ средств автоматизации описания атак на средства машинного обучения и оценки рисков их реализации

Раздел 2. Разработка модулей системы по оценке рисков и генерации мер противодействия для атак на средства машинного обучения

Раздел 3. Управление информационной системой

Краткая характеристика полученных результатов

Выполнен обзор открытых баз и информационных систем описания атак на средства машинного обучения, выявлены пробелы в формализации и представлении данных;

Сформирован метод структурированного представления сведений об атаках, предложена модульная архитектура информационной системы, объединяющая функции агрегирования данных об уязвимостях, оценки рисков и генерации мер противодействия;

Разработан программный комплекс, реализующий автоматизированный сбор, нормализацию и агрегирование данных об уязвимостях, оценку рисков и генерацию мер противодействия.

Автор ВКР _____ Кульшин Дмитрий Вячеславович
(подпись)

СОДЕРЖАНИЕ

Введение 8
1 Анализ средств автоматизации описания атак на средства машинного обучения и оценки рисков их реализации 11
1.1 Обзор существующих средств 11
1.1.1 Классификация средств 13
1.1.2 Критерии оценки средств 13
1.2 Архитектура и ключевые компоненты средств 14
1.2.1 Модель тактик и техник 14
1.2.2 Инструменты для моделирования атак 15
1.3 Сравнительный анализ и задачи дальнейшей автоматизации 15
2 Разработка модулей системы по оценке рисков и генерации мер противодействия 23
2.1 Автоматизация агрегирования данных об уязвимостях средств машинного обучения 23
2.1.1 Структура данных 24
2.1.2 Технологическое обеспечение 26
2.1.3 Функциональные библиотеки 27
2.1.4 Лингвистическое обеспечение 31
2.1.5 Алгоритмы сбора и обработки данных 31
2.2 Автоматизация оценки рисков атак на средства машинного обучения 40
2.2.1 Структура данных 42
2.2.2 Технологическое обеспечение 42
2.2.3 Функциональные библиотеки 43
2.2.4 Лингвистическое обеспечение 44
2.2.5 Алгоритм оценки риска и составления сценариев 45
2.3 Автоматизация сопоставления уязвимостей с техниками 48
2.3.1 Структура данных 49
2.3.2 Технологическое обеспечение 50
2.3.3 Функциональные библиотеки 50

2.3.4 Лингвистическое обеспечение 52
2.3.5. Алгоритм сопоставления уязвимостей с техниками 52
2.4 Алгоритмизация генерации мер противодействия 57
2.4.1 Структура данных 58
2.4.2 Технологическое обеспечение 58
2.4.3 Функциональные библиотеки 59
2.4.4 Лингвистическое обеспечение 59
2.4.5 Алгоритм генерации мер противодействия 60
3 Управление информационной системой 65
3.1 Руководство пользователя 65
3.1.1 Установка и первоначальная настройка 66
3.1.2 Работа с интерфейсом 66
3.2 Руководство для разработчиков 73
3.2.1 Архитектура приложения 73
3.2.2 API и внутренние интерфейсы 76
3.3 Рекомендации по расширению функционала информационной системы 77
3.3.1 Новые модули и функции 77
3.3.2 Улучшение пользовательского интерфейса 78
3.4 Обеспечение информационной безопасности разработанной системы 79
Заключение 81
Список литературы 83
Приложение А 89
Приложение Б 90

ВВЕДЕНИЕ

Современное информационное пространство характеризуется высокой динамикой возникновения новых угроз [1-9]. Поэтому для обеспечения безопасности требуется постоянный мониторинг угроз, их оценка и оперативное реагирование [10].

Технологии машинного обучения активно внедряются в различные сферы человеческой деятельности, включая безопасность, медицину, финансы и автоматизацию производственных процессов [11-12]. Однако их широкое внедрение сопровождается ростом числа атак, направленных на эксплуатацию уязвимостей алгоритмов машинного обучения и их реализации [13-16].

Исходя из вышесказанного, одной из важных задач обеспечения информационной безопасности является необходимость систематизации знаний об атаках на средства машинного обучения, а также создание инструмента для построения сценариев этих атак и анализа рисков.

Актуальность темы исследования обусловлена наличием противоречий между:

– стремительно растущим ареалом внедрения средств машинного обучения и отсутствием автоматизированного метода представления сведений об атаках на средства машинного обучения;

– необходимостью в алгоритме функционирования информационной системы автоматизированного сбора данных об уязвимостях машинного обучения, оценки рисков и выработки мер противодействия для атак, направленных на средства машинного обучения и отсутствием такого алгоритма;

– необходимостью программной реализации информационной системы, обеспечивающей агрегацию данных об уязвимостях, оценку риска и выработку мер противодействия для атак, направленных на средства машинного обучения и отсутствием такого программного обеспечения.

Объектом исследования являются информационные автоматизированные системы, включающие средства машинного обучения.

Предметом исследования является разработка алгоритмического и программного обеспечения оценки рисков и формирования мер противодействия, для атак на средства машинного обучения.

Целью исследования является повышение защищенности информационных автоматизированных систем, включающих средства машинного обучения, за счет разработки алгоритмического и программного обеспечения оценки рисков и формирования мер противодействия, для атак на средства машинного обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести сравнительный анализ существующих информационных ресурсов, содержащих сведения об атаках на средства машинного обучения на предмет наличия недостатков функциональных особенностей;

– разработать алгоритм функционирования и взаимодействия модулей, обеспечивающих агрегацию данных об уязвимостях, оценку рисков и выработку мер противодействия для атак, направленных на средства машинного обучения;

– реализовать программные модули, обеспечивающие агрегирование данных об уязвимостях, оценку риска и генерацию мер противодействия для атак, направленных на средства машинного обучения.

В процессе решения задач исследования были получены следующие результаты:

– автоматизированный метод представления данных об атаках на средства машинного обучения;

– алгоритм взаимодействия и функционирования модулей информационной системы реализующей агрегацию данных об уязвимостях средств машинного обучения, оценку рисков и выработку мер противодействия, для атак на средства машинного обучения;

– программное обеспечение, реализующее агрегацию данных об уязвимостях средств машинного обучения, оценку рисков и выработку мер противодействия, для атак на средства машинного обучения.

Новизна полученных результатов заключается в том, что:

– в отличие от аналогов, разработан метод структурированного и автоматизированного представления информации об атаках на средства машинного обучения;

– в отличие от аналогов, предложен алгоритм, объединяющий функции агрегации уязвимостей, оценки рисков и формирования мер противодействия в контексте защиты средств машинного обучения;

– в отличие от аналогов, разработан программный комплекс, впервые интегрирующий весь цикл обработки данных об уязвимостях средств машинного обучения – от сбора до противодействия.

Теоретическая значимость результатов, полученных в ходе исследования:

– обеспечивает формальную основу для систематизации и классификации атак на модели машинного обучения с учетом особенностей алгоритмов и архитектур;

– позволяет построить адаптивную систему мониторинга и реагирования, способную автоматически анализировать потоковые данные об уязвимостях в средствах машинного обучения и формировать меры в реальном времени;

– демонстрирует применимость комплексных архитектурных решений для безопасности в области средств машинного обучения.

Практическая значимость результатов, полученных в ходе исследования:

– метод позволяет упростить интеграцию сведений об атаках в системы управления безопасностью средств машинного обучения и ускорить анализ инцидентов;

– алгоритм может быть внедрен в существующие SIEM/SOAR-решения для повышения их эффективности при обнаружении угроз средствам машинного обучения;

– предложенное программное обеспечение обеспечивает автоматизированную защиту средств машинного обучения от специфических угроз за счет оценки рисков и рекомендаций по реагированию.

1 Анализ средств автоматизации описания атак на средства машинного обучения и оценки рисков их реализации

1.1 Обзор существующих средств

В сфере современной информационной безопасности инструменты кажутся исчерпывающими по своим возможностям. Они представляют собой матрицы тактик и техник, дают доступ к их описаниям и часто включают переводы.

Тем не менее, рассматриваемые решения преимущественно выступают в роли справочников, задача которых сводится к предоставлению пользователю сведений о техниках и потенциальных методах противодействия.

Если же необходимо объединить разрозненные данные в непрерывный цикл, ситуация кардинально меняется. Ни один из исследуемых ресурсов не занимается агрегацией данных об уязвимостях из внешних источников. Сведения об уязвимостях, оценках критичности и ущербе по умолчанию не собираются напрямую. Пользователю необходимо выполнять сбор этих данных самостоятельно. Процесс сбора, нормализации и хранения данных об уязвимостях для последующего анализа остается за рамками возможностей средств описания атак.

Тепловые карты, градации по уровню использования техник и интеграция с реальными инцидентами, реализованные, например, в MITRE ATLAS [17], выглядят полезно, но не обеспечивают полноценной модели для количественной оценки рисков.

Критически важным аспектом является автоматизированное построение сценариев атак.

Наконец, при попытке сгенерировать меры противодействия, пользователь получает лишь перечень рекомендаций и вынужден вручную сопоставлять уязвимости с техниками, разыскивая в соответствующих разделах подходящие варианты. Однако автоматическая генерация мер, учитывающая сценарий атаки, конкретные техники и эксплуатируемые ими уязвимости, отсутствует.

Развитие технологий машинного обучения открывает новые горизонты для автоматизации процессов и повышения эффективности принятия решений, но в то

же время влечет за собой и усиление рисков – злоумышленники учатся эксплуатировать уязвимости моделей, создавать целевые атаки и обходить встроенные механизмы защиты. В связи с этим возникает необходимость в специализированных инструментах, способных не только собирать информацию о уязвимостях, но и автоматически оценивать вероятность их успешного проведения, формировать сценарии эксплуатации и предлагать меры противодействия.

На текущий момент наиболее распространенными платформами для моделирования и документирования тактик и техник атак выступают фреймворки MITRE ATT&CK [18] и MITRE ATLAS.

MITRE ATT&CK представляет собой глобальную базу знаний о тактиках и техниках, основанную на практических наблюдениях. MITRE ATT&CK фокусируется на классических угрозах IT-инфраструктуре.

MITRE ATLAS – это доступная база знаний о тактиках и техниках, характерных для систем, использующих методы машинного обучения.

Интерактивный ATLAS NAVIGATOR [19] предоставляет удобный веб-интерфейс для создания слоев, фильтрации техник по типу данных и модели, отображения тепловых карт вероятностей переходов между приемами и экспорта сценариев в формате JSON. Он позволяет группировать техники атак против моделей, строить цепочки действий и настраивать собственные «карты угроз».

Отечественный сервис от «Код Безопасности» [20] предлагает веб-интерфейс для интерактивного моделирования сценариев вторжений, базируясь на матрице ATT&CK версии 16.1 и используя машинное обучение для выявления взаимосвязей между техниками, опираясь на данные четырехсот реальных атак за последний год. Анализ полученных данных дает возможность спрогнозировать действия нарушителей при проведении атаки, а также построить комплексную систему защиты. Пользователю предлагается не просто набор техник, но и визуализация их последовательностей с возможностью агрегации и детализации по подтехникам, что упрощает оценку рисков на каждом этапе атаки и последующий выбор мер защиты.

Positive Technologies интегрировали ATT&CK v15.1 [21] в свою систему, отображая панель покрытия техник на уровне 80 % и демонстрируя, какие именно

техники детектируются их решениями. Статус "покрыто" или "не покрыто" иллюстрируется посредством счетчика выполненных проверок по каждой технике, что позволяет оценить эффективность конкретного набора средств в различных тактических категориях.

Несмотря на то, что все три ресурса обеспечивают удобный поиск и визуализацию, их возможности по автоматической агрегации обширного перечня уязвимостей, количественной оценке рисков и генерации рекомендаций по защите остаются ограниченными.

1.1.1 Классификация средств

Современные решения для анализа угроз и моделирования атак можно разделить на несколько функциональных групп, каждая из которых решает специфические задачи и дополняет общую картину обеспечения безопасности. Первый класс объединяет справочники знаний – системы, выступающие централизованным хранилищем техник и уязвимостей, снабженным подробными описаниями контекста применения и индикаторами компрометации. Во втором классе находятся интерактивные навигаторы: они предоставляют визуальные представления фреймворков (ATT&CK или ATLAS), позволяют настраивать отображение тактик и техник, накладывать дополнительные слои данных. Третья категория включает платформы сценарного моделирования, воспроизводящие возможные цепочки атак: такие решения автоматически генерируют последовательности действий злоумышленника.

1.1.2 Критерии оценки средств

При сравнении подобного рода систем необходимо обратить внимание на следующие ключевые аспекты. Во-первых, полнота и актуальность базы знаний, определяющая глубину проработки техник и уязвимостей: важно, чтобы данные обновлялись регулярно и включали сведения о новых атаках на средства машинного

обучения их алгоритмы. Во-вторых, возможности агрегирования информации, то есть умение собирать данные из разных источников и представить единый, унифицированный формат для анализа. И, наконец, уровень автоматизации управления риском: наличие модулей для количественной оценки, а также механизмов автоматического формирования рекомендаций по реагированию.

1.2 Архитектура и ключевые компоненты средств

Общая архитектура рассматриваемых систем включает три уровня: репозиторий тактик и техник MITRE ATT&CK или MITRE ATLAS, модуль аналитики, машинное обучение для выявления связей между событиями, и презентационную часть информационной системы.

Для «Кода Безопасности» репозиторий представляет собой датасет из четырёхсот реальных сценариев, аннотированных по матрице ATT&CK, аналитика опирается на вероятностные графовые модели, а визуализация реализована в виде динамической матрицы с режимами детализации. В случае Positive Technologies основной фокус смещен на интеграцию с продуктами мониторинга: здесь вместо открытого датасета используется эмпирическая статистика «покрытия» техник, а аналитический слой оценивает лишь готовность решений Positive Technologies к отражению известных техник. ATLAS Navigator отличается наличием репозитория ATLAS, включающего техники ориентированные на средства машинного обучения, и тепловых карт, показывающих частоту упоминаний приёмов.

1.2.1 Модель тактик и техник

Во всех трёх системах модель тактик и техник в основе своей соответствует оригинальным фреймворкам MITRE ATT&CK или MITRE ATLAS, однако отличаются по глубине и детализации. «Код Безопасности» обеспечивает агрегацию на уровне основных техник и подклассов, позволяя отслеживать отдельные подтехники, например, разнообразие PowerShell-скриптов в рамках T1059. Вариант

от Positive Technologies дополняет каждую технику счётчиком «покрыто/всего». ATLAS Navigator вводит новые тактики «ML Model Access» и «ML Attack Staging», что позволяет анализировать атаки на системы машинного обучения, однако не включает централизованный реестр уязвимостей и требует ручного форматирования.

1.2.2 Инструменты для моделирования атак

Сервис «Код Безопасности» предлагает два режима моделирования: базовое моделирование – показывает техники, которые связаны с выбранной техникой. Эти техники могут быть реализованы хакером перед или после выбранной техники. Иными словами, это топ часто встречающихся связей для выбранной техники. показывает техники, которые выполняются в последовательной цепочке. Это моделирование показывает, какие смежные техники должен выполнить хакер для реализации выбранной техники. Оба типа моделирования можно строить как с учётом Алгоритмы машинного обучения выявляют вероятностные переходы между техниками на основе исторических данных, что позволяет моделировать наиболее вероятные пути развития атаки.

Платформа Positive Technologies, в свою очередь, интегрируется непосредственно с продуктами мониторинга и защиты, автоматически обновляя статистику покрытия и визуализируя те техники, которые ещё не «закрыты» существующими решениями.

ATLAS Navigator дает максимальную гибкость: пользователь может загружать собственные JSON-слои техник, строить тепловые карты по частоте упоминаний и экспортировать данные в STIX/Excel для последующей автоматической обработки.

1.3 Сравнительный анализ и задачи дальнейшей автоматизации

Средство от «Код Безопасности» представляет собой веб-инструмент, позволяющий моделировать потенциальные действия злоумышленников на ИТ-инфраструктуру с опорой на общепризнанный фреймворк MITRE ATT&CK.

Основой системы служит обширный датасет, составленный на основе около четырёх сотен реальных атак, собранных за год. Благодаря этому модель отражает тенденции и приемы, используемые в реальных условиях. Регулярные обновления гарантируют актуальность выводов и рекомендаций.

При первом взаимодействии со средством пользователь может выбрать отрасль деятельности предприятия, что позволяет адаптировать модель к специфике угроз. Так, например, техники популярные в финансовом секторе, могут не применяться в образовательной сфере, и наоборот. На рисунке 1 показаны смежные техники для техники «T1078» – «действительные учетные данные» в финансовой отрасли.

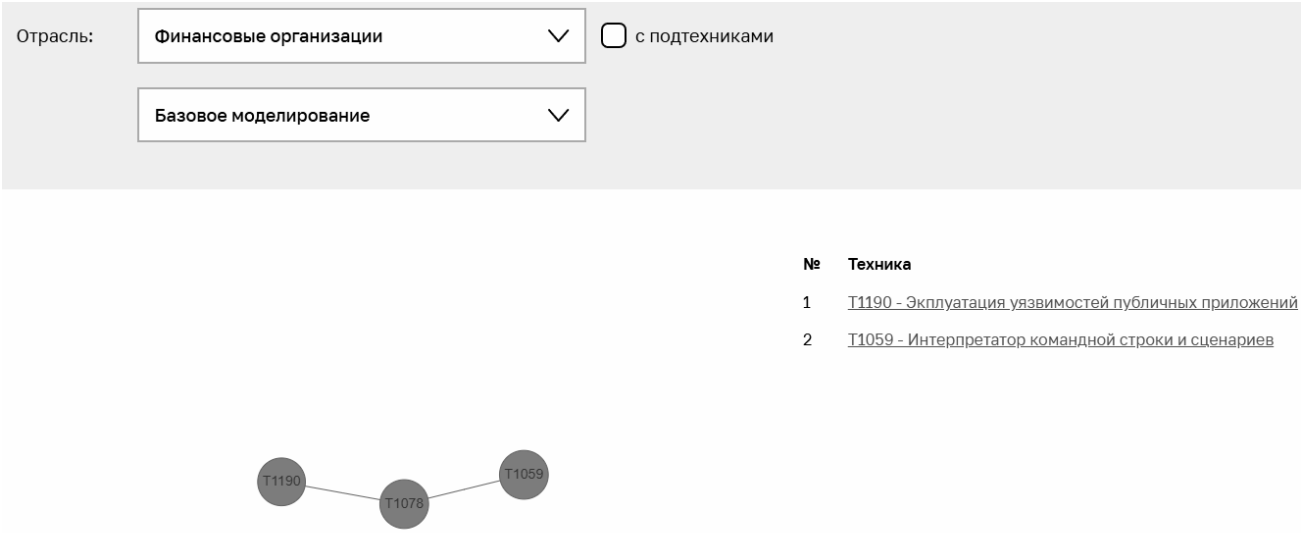


Рисунок 1 – Окно отображения смежных техник для техники «T1078»

Для финансовой области смежными являются две техники:

– «T1190» – эксплуатация уязвимостей публичных приложений;

– «T1059» – интерпретатор командной строки и сценариев.

Для образовательной сферы набор связанных техник отличается. Это представлено на рисунке 2.

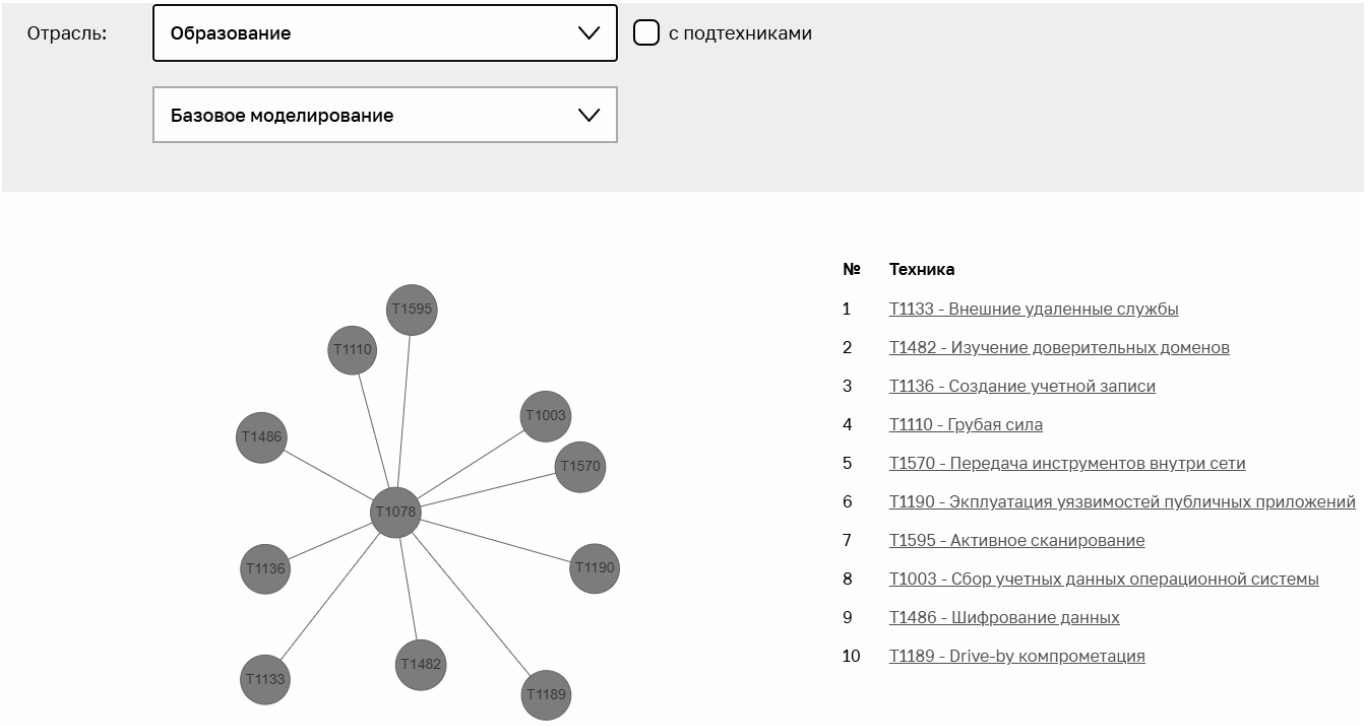


Рисунок 2 – Окно отображения смежных техник для техники «T1078»

Для сферы образования смежными являются уже 10 техник:

– «T1133» – внешние удаленные службы;

– «T1482» – изучение доверительных доменов;

– «T1136» – создание учетной записи;

– «T1110» – грубая сила;

– «T1570» – передача инструментов внутри сети;

– «T1190» – эксплуатация уязвимостей публичных приложений;

– «T1595» – активное сканирование;

– «T1003» – сбор учетных данных операционной системы;

– «T1486» – шифрование данных;

– «T1189» – drive-by компрометация.

Система автоматически формирует список наиболее часто используемых в выбранной отрасли техник, представляя их в порядке убывания их распространенности. Такой предварительный обзор позволяет сосредоточить внимание на наиболее критичных аспектах улучшения защиты, избегая необходимости глубокого изучения каждого элемента матрицы.

Для более глубокого анализа пользователь может выбрать конкретную технику из матрицы MITRE ATT&CK. Система выделяет связанные с ней техники, базирующиеся на методах машинного обучения, и предлагает два режима визуализации:

– базовое моделирование, демонстрирующее ближайшие связи;

– расширенное, позволяющее построить цепочку последовательных действий.

При этом возможна работа, как с базовыми техниками, так и с учетом подтехник.

Интерфейс сервиса включает в себя русифицированную матрицу MITRE ATT&CK, интерактивные графики и панели с детальным описанием каждой техники. Сервис демонстрирует свои возможности в формате видео и предусматривает механизм обратной связи для внесения уточнений и доработок. Версия фреймворка MITRE ATT&CK, используемая в системе, соответствует релизу 16.1, что обеспечивает соответствие последним рекомендациям в области безопасности. Пользователи могут обращаться в круглосуточную службу поддержки, получать консультации по электронной почте или телефону, а также интегрировать результаты моделирования в собственные процессы управления рисками и аудита безопасности.

Сервис на базе веб-интерфейса от Positive Technologies представляет собой динамическую адаптацию матрицы MITRE ATT&CK, отображающую степень охвата различных техник решениями данного вендора. При загрузке страницы сразу отображается общая метрика «Покрытие продуктами 80%» и номер используемой версии фреймворка ATT&CK, на данный момент 15.1. Ниже пользователя встречает привычная матрица ATT&CK, сгруппированная в виде интерактивных вкладок. При нажатии на любую из них матрица переформатируется, отображая только подтехники и техники, релевантные выбранной фазе атаки, с указанием доли покрытых продуктов. Дополнительный переключатель «Только покрытые техники» позволяет скрыть из вида все элементы, не поддерживаемые решениями отечественных вендоров. Все наименования техник снабжены гиперссылками на

оригинальные сведения из MITRE ATT&CK, что облегчает детальное изучение описаний, индикаторов и рекомендаций по детектированию в системах SIEM и EDR.

MITRE ATLAS NAVIGATOR является надстройкой над известным MITRE ATT&CK NAVIGATOR, адаптированной под специфику угроз, связанных со средствами машинного обучения и основанной на базе MITRE ATLAS. При загрузке веб-интерфейса перед пользователем предстает привычная матрица. Однако, в сравнении с классической матрицей ATT&CK, здесь добавлены новые тактики, отражающие особенности атак на системы машинного обучения. Благодаря этому расширению пользователи получают визуальный инструмент, позволяющий одновременно видеть и традиционные киберугрозы, и специфические атаки на автоматизированные системы, использующие средства машинного обучения.

Самое примечательное в ATLAS NAVIGATOR – возможность наложения цветовых тепловых карт, отображающих частоту упоминания каждой техники в реальных инцидентах, описанных в ATLAS. Чем чаще техника фиксировалась в отчетах, тем ярче она подсвечивается, что дает оперативное понимание, какие техники наиболее актуальны в текущем ландшафте угроз, связанных со средствами машинного обучения. Эти тепловые карты не просто акцентируют «горячие» зоны атак, но и помогают структурировать приоритеты при построении средств обнаружения и реагирования.

Гибкость ATLAS NAVIGATOR проявляется и в работе со слоями. Каждый слой представляет собой отдельный набор техник и связей, который можно загрузить через специализированный JSON-файл или создать вручную, работая в интерфейсе. Пользователь способен спроектировать слой, отражающий, конкретные действия определенной группы злоумышленников, а затем наложить его поверх стандартного ATLAS-слоя, выявив пересечения и пробелы в защите.

Полученный слой сохраняется в виде URL-ссылки, что позволяет делится результатами анализа с коллегами или интегрировать визуализацию в документацию и отчеты без потери интерактивности.

Помимо визуализации, NAVIGATOR предоставляет экспорт данных: с помощью модифицированных скриптов «STIX-to-Excel» можно загрузить полный

набор данных в табличные файлы XLSX. Репозиторий содержит готовые примеры и инструкции по настройке, что позволяет аналитикам легко интегрировать описания техник в отчеты или загружать их в собственные инструменты. Такое сочетание интерактивного веб-интерфейса и возможностей обработки данных делает NAVIGATOR удобным как для оперативной разведки, так и для глубоких исследований и аудитов.

Инструмент ориентирован на совместную работу и обмен знаниями. Любой созданный слой доступен по уникальной ссылке, а встроенные функции копирования конфигурации позволяют быстро распределять анализ между разными подразделениями. Благодаря открытой лицензии и размещению кода на платформе «GitHub», организации могут самостоятельно дорабатывать как визуальную часть, так и логику фильтрации, адаптируя NAVIGATOR под внутренние процессы управления рисками и соответствия регуляторным требованиям.

При прикладном сравнительном анализе представленных средств выявляются существенные различия в их целевой направленности и функциональных возможностях, а также недостатки каждого из них по отношению к целям исследования.

Сервис от «Код безопасности» ориентирован на моделирование последовательностей техник по классической матрице MITRE ATT&CK, что реализуется посредством построения связей «до-после» и формирования цепочек техник. Несмотря на то, что он базируется на датасете из четырехсот реальных сценариев, он не затрагивает специфику атак на средства машинного обучения и не предоставляет механизмов агрегирования сведений об уязвимостях данных средств. Более того, в нем отсутствуют встроенные алгоритмы количественной оценки риска и автоматической генерации рекомендаций по реагированию. Пользователю предлагается самостоятельно анализировать визуализированные сценарии и на их основе разрабатывать меры противодействия.

Решение Positive Technologies показывает степень покрытия техник ATT&CK продуктами вендора, что актуально для аудита существующих защитных средств, но не затрагивает моделирование атак, и тем более не учитывает новые векторы,

специфичные для средств машинного обучения. Вся информация подается без глубокого анализа сценариев, оценки вероятности реализации уязвимости и отсутствует генерация рекомендаций по адаптации правил SIEM/EDR или корректировке политик безопасности на основе рисков.

MITRE ATLAS NAVIGATOR привносит в ATT&CK подход, сфокусированный на угрозах, связанных со средствами машинного обучения, включая расширенные тактики и «тепловые карты», отображающие частоту упоминания в инцидентах, описанных в ATLAS. Однако интерфейс доступен только на английском языке, а сам инструмент служит для визуальной приоритизации, но не содержит встроенной базы уязвимостей средств машинного обучения, не предлагает автоматизированной оценки рисков и не генерирует конкретных мер противодействия или политики противодействия на основе обнаруженных пробелов в защите.

Для более наглядного и структурированного сопоставления ключевых характеристик рассмотренных инструментов целесообразно представить результаты в виде сравнительной таблицы 1.

Таблица 1 – Сравнительный анализ средств

Критерий	«Код безопасности»	Positive Technologies	ATLAS NAVIGATOR
Представление на русском языке	Полностью на русском	Полностью на русском	Только английский
Агрегирование сведений об уязвимостях	Отсутствует	Отсутствует	Отсутствует
Моделирование сценариев атак	Цепочки техник, базовое и расширенное моделирование	Нет	Без готовых сценариев
Оценка риска	Нет	Нет	Визуализация без оценки значений

Продолжение таблицы 1

Критерий	«Код безопасности»	Positive Technologies	ATLAS NAVIGATOR
Генерация рекомендаций	Нет	Нет	Нет
Приоритизация техник	Ручной выбор по частоте в отрасли	По степени покрытия продуктов	Тепловые карты по статистике ATLAS

Неполнота информационных возможностей вышеупомянутого инструментария обуславливает необходимость уточняющих и систематизирующих исследований в данном направлении [22-26].

Анализ доступных решений выявил недостаток комплексности в сфере безопасности машинного обучения. Отсутствует интегрированный подход, который включал бы в себя не только сбор сведений об известных уязвимостях и их систематизацию, но и моделирование возможных атак с последующей количественной оценкой риска. Кроме того, автоматическая генерация мер противодействия не предусмотрена. Это создает острую необходимость в создании платформы, объединяющей указанные выше три ключевых элемента в единую систему. Лишь благодаря применению такого комплексного подхода можно будет не просто реагировать на уже обнаруженные уязвимости, но и оценивать риск, а также предлагать автоматизированные меры противодействия.

2 Разработка модулей системы по оценке рисков и генерации мер противодействия

2.1 Автоматизация агрегирования данных об уязвимостях средств машинного обучения

Модуль агрегации данных об уязвимостях средств машинного обучения призван централизовать сбор, хранение и предварительную обработку информации о найденных уязвимостях. Модуль поддерживает интеграцию со следующими ресурсами:

– банк данных угроз (БДУ) ФСТЭК России [27] – отечественный реестр, содержащий подробные сведения об уязвимостях и рекомендации по их устранению;

– National Vulnerability Database (NVD) от NIST [28] – международный каталог CVE-записей с подробной информацией;

– huntr.com [29] – сообщество, которое предоставляет исследователям единое место для отправки сообщений об уязвимостях, связанных со средствами машинного обучения.

Модуль обеспечивает унификацию форматов и фильтрацию, что существенно упрощает дальнейший анализ и оценку рисков. Задачи, решаемые с помощью данного модуля:

– сбор данных, с поддержкой запросов к API;

– получение оценки EPSS [30];

– получение оценки AIVSS [31];

– обработка отдельных форматов. JSON для NVD от NIST, XLSX для БДУ ФСТЭК России, HTML-страницы для huntr.com;

– приведение сведений в единый формат;

– загрузка в СУБД для дальнейшей обработки.

AIVSS – это стандартизированный фреймворк для количественной оценки уязвимостей средств машинного обучения [32].

Данный фреймворк был создан OWASP [33]. Его цель – адаптировать существующие методы анализа уязвимостей к средствам машинного обучения, в особенности к большим языковым моделям и облачным окружениям. В отличие от стандартных подходов, таких как CVSS он фокусируется на специфических рисках, характерных для средств машинного обучения. Это непрозрачность работы модели, динамическое изменение ее параметров в процессе дообучения. Кроме того, AIVSS учитывает этические и социальные последствия некорректных решений, принимаемых в системах, использующих средства машинного обучения.

В связи с тем, что модуль взаимодействует с разными источниками, необходимо разделить процесс получения данных об уязвимостях на несколько компонентов, каждый из которых взаимодействует со своим источником.

2.1.1 Структура данных

Способ получения данных различен для каждого из приведенных выше источников. При работе с NVD от NIST модуль обрабатывает и трансформирует сложный JSON-объект, который возвращает API. Данный JSON-объект представляет собой словарь с ключом «vulnerabilities», значением которого является список записей. Каждая запись содержит идентификатор уязвимости, описание, метрики CVSS [34], тип ошибки CWE [35].

При работе с БДУ ФСТЭК России нет возможности получать данные напрямую через API. Но данный ресурс позволяет скачать сведения об уязвимостях в формате XLSX. В данной таблице содержатся идентификаторы уязвимостей, их описания, метрики CVSS, типы ошибок CWE, а также возможные меры по устранению уязвимости.

Ресурс huntr.com не предоставляет возможности взаимодействия с API, также нет возможности загрузить информацию об уязвимостях в каком-либо формате. Поэтому при работе с ним взаимодействие происходит напрямую с HTML-кодом страницы и необходимые данные извлекаются из него.

Для получения оценки EPSS, модуль отправляет запрос GET запрос к API и обрабатывает и трансформирует полученный JSON-объект. Данный объект представляет собой словарь, ключевым полем которого является «epss», это поле хранит необходимое значение.

Этот компонент необходим по следующим причинам:

– повышение оперативности обнаружения, так как EPSS предоставляет статистически обоснованные оценки вероятности эксплуатации уязвимостей в ближайшем будущем. Автоматический компонент позволит своевременно получать актуальные данные;

– улучшение приоритизации. Включив EPSS-оценку в процесс оценки уязвимостей, возможно учитывать не только традиционную CVSS-оценку, но и вероятность реальной эксплуатации. Это даст более точную картину приоритета исправления уязвимостей;

– минимизация человеческого фактора и ошибок. Ручной сбор данных с нескольких источников – трудоемкий и подверженный ошибкам процесс. Автоматизированный компонент гарантирует единообразие и полноту собираемой информации, исключая неверный ввод или пропуск важной детали.

В рамках разработки модуля агрегации данных был выбран SQLite [36] в качестве хранилища данных об уязвимостях по следующим причинам. Во-первых, SQLite представляет собой встроенную СУБД, не требующую развертывания отдельного серверного процесса: вся база хранится в одном файле, что упрощает установку и переносимость решения. Во-вторых, для данных объемом до 10-20 ГБ и относительно невысокой частоты одновременных подключений производительность чтения и записи SQLite оказывается вполне достаточной. Благодаря поддержке транзакций ACID и механизмам журналирования WAL обеспечивается высокая надежность и целостность данных даже при сбоях. Структура базы включает в себя таблицу cves, которая включает в себя следующие поля:

– cve_id: идентификатор уязвимости;

– cwe: тип ошибки CWE;

– description: описание уязвимости;

– technique: метка техники MITRE ATLAS, к которой относится уязвимость;

– cvss3: вектор CVSS версии 3;

– cvss3score: оценка CVSS версии 3;

– cvss4: вектор CVSS версии 4.0;

– cvss4score: оценка CVSS версии 4.0;

– epss: значение оценки EPSS;

– aivss_score: значение оценки AIVSS;

– final_score_sg: средневзвешенная оценка критичности уязвимости;

– cia: значение ущерба уязвимости;

– have_exploit: флаг указывающий на существование эксплоита.

Таким образом, SQLite обеспечивает простоту развертывания, надёжность и достаточную производительность.

2.1.2 Технологическое обеспечение

В компоненте, связанном с NVD, реализована автоматическая интеграция с публичным API. Отправная точка – это URL-адрес API, задающий конечную точку REST-сервиса для получения информации о CVE. За счёт использования HTTP-запросов и многопоточной модели, обеспечивается масштабируемый и отказоустойчивый доступ к данным NVD. Данный компонент выполняет следующие функции:

– подключение к публичному API NVD;

– получение записей об уязвимостях;

– разбор структуры CVE-объектов и приведение их к единому формату;

– отбор уязвимостей относящихся к средствам машинного обучения;

– извлечение метрик CVSS, типа ошибки CWE;

– сохранение в СУБД.

Компонент ответственный за загрузку и обработку данных из БДУ ФСТЭК России предназначен для автоматизированного извлечения и предварительной обработки табличных данных в формате XLSX. В модуле реализован механизм

потоковой загрузки файла содержащего сведения об уязвимостях. Данный компонент выполняет следующие функции:

– потоковая загрузка официального XLSX файла с уязвимостями;

– обработка XLSX файла для получения записей об уязвимостях;

– отбор уязвимостей относящихся к средствам машинного обучения;

– извлечение метрик CVSS, типа ошибки CWE;

– сохранение в СУБД.

В компоненте, отвечающем за сбор данных с портала huntr.com реализована автоматизация работы с веб-интерфейсом и HTML-страницами. В основе лежит движок Chrome, который по заданному URL-адресу раздела с уязвимостями последовательно подгружает новые записи и извлекает из них сведения об уязвимостях. Данный компонент выполняет следующие функции:

– подключение к стартовой странице huntr.com;

– обработка HTML-страницы для получения записей об уязвимостях;

– извлечение сведений об уязвимостях;

– сохранение в СУБД.

В компоненте, отвечающем за сбор данных об оценке EPSS реализована интеграция с публичным API. Начальная точка – это URL-адрес API, на который отправляется GET запрос, содержащий идентификатор уязвимости. После, из полученного объекта извлекается оценка EPSS. Ключевые функции разработанного компонента:

– запрос актуальных EPSS-оценок через официальное API;

– сохранение в СУБД.

2.1.3 Функциональные библиотеки

Для разработки компонента, работающего с NVD, были использованы следующие библиотеки.

Requests – это python-библиотека, ориентированная на выполнение HTTP-запросов. Она призвана упростить стандартную библиотеку urllib, предоставляя

более удобный интерфейс для работы с HTTP. Применяется для интеграции с внешними API, тестирования REST-сервисов. Особенностями данной библиотеки являются:

– удобная работа с JSON;

– гибкая настройка сетевого взаимодействия;

– автоматическая конвертация кодировок;

– поддержка таймаутов и повторных попыток;

– простой механизм файловой загрузки.

Json – стандартная библиотека python, которая обеспечивает средства для кодирования и декодирования данных в формате JSON. JSON – легкий текстовый формат обмена данными, широко использующийся в веб-разработке и межпроцессном взаимодействии. Применяется для обмена данными в REST API, конфигурационных файлах, межъязыкового взаимодействия. Данная библиотека имеет ряд особенностей:

– формат JSON рассчитан на минимальный синтаксис, что упрощает чтение и отладку как для человека, так и для машин;

– высокая скорость кодирования и декодирования;

– гибкость настройки вывода;

– совместимость с большинством веб-API и межпроцессных протоколов делает JSON стандартным выбором для обмена данными в распределенных системах.

Concurrent.futures – библиотека python для унификации и упрощения параллельного, а именно многопоточного и многопроцессного, исполнения задач. Применяется для задач ограниченного ввода-вывода, в частности сетевых запросов и операций с файлами. И тяжелых вычислений связанных с CPU [37]. Ограничениями данной библиотеки являются:

– накладные расходы на кодирование аргументов и результатов при использовании процессов;

– потенциальное состояние гонки при некорректной работе с общим состоянием в потоках;

– сложности отладки и профилирования параллельного кода.

Для разработки компонента взаимодействующего с БДУ ФСТЭК России помимо библиотеки requests используются следующие библиотеки.

Pandas – библиотека python, ориентированная на удобную работу с табличными и временными данными. Благодаря своей производительности и богатому API стала стандартом для анализа данных в экосистеме python. Применяется для научных исследований, анализа данных, финансовых расчетов, подготовки данных для машинного обучения. Особенностями данной библиотеки являются:

– высокая производительность, благодаря реализации на C и Cython многие операции, такие как фильтрация, агрегация, объединение, работают на скоростях, близких к скоростям низкоуровневых языков;

– широкий набор интерфейсов ввода-вывода, а именно встроенная поддержка CSV, XLSX, SQL, JSON и других форматов, что позволяет легко интегрироваться со многими хранилищами данных;

Re – стандартная библиотека python. Обеспечивает реализацию регулярных выражений для текстового поиска и обработки. Данная библиотека построена поверх высокопроизводительного движка основанного на языке C, что обеспечивает быстрое сопоставление шаблонов. Применяется для валидации и разбора текстовых форматов, преобразования текстов, синтаксического анализа. Особенностями данной библиотеки являются:

– полное соответствие Perl-совместимому синтаксису, то есть поддержка групп, квантификаторов, обратных ссылок, опережающих и ретроспективных проверок, что позволяет выражать практически любые шаблоны;

– благодаря движку на языке C сопоставление работает быстрее, чем многие чисто интерпретируемые реализации;

– возможность заранее компилировать регулярное выражение и многократно его использовать;

– флаги модификации поведения, которые дают гибкость при написании и отладке сложных шаблонов;

Os – стандартная библиотека python, которая предоставляет высокоуровневый интерфейс к функциональности операционной системы, причем его поведение как правило не зависит от операционной системы. Особенностями данной библиотеки являются:

– единый API для Windows, Linux и macOS, который автоматически адаптируется под особенности целевой ОС;

– большинство функций зеркально повторяют POSIX-интерфейс;

– низкоуровневый и высокоуровневый доступ;

– работа с окружением, а именно простой доступ к переменным окружения и смена текущей директории.

Для работы компонента, ответственного за агрегацию данных были использованы следующие библиотеки.

BeautifulSoup – библиотека python, разработанная для парсинга и навигации по документам HTML и XML. Призвана упростить извлечение данных из разметки. Представляет единый интерфейс для работы сразу с несколькими низкоуровневыми парсерами. Особенностями данной библиотеки являются:

– автоматически исправляет незакрытые теги и некорректную вложенность, что важно при работе с невалидными страницами;

– поддержка поиска по имени тега, атрибутам, текстовому содержимому, CSS-селекторами и комбинированных фильтров;

– минималистический и интуитивный API позволяет быстро начать разработку;

– выбор оптимального движка в зависимости от требований к скорости или точности разбора.

Selenium – фреймворк для автоматизированного управления веб-браузером. Поддерживает несколько языков и взаимодействует с браузерами через WebDriver-протокол [38]. Ключевыми особенностями являются:

– единый API для Chrome, Firefox, Safari, Edge и других, что упрощает кросс-браузерное тестирование;

– полная эмуляция поведения пользователя, в том числе загрузка JavaScript, работа с AJAX, выполнение событийных обработчиков;

– способность взаимодействовать с элементами, недоступными для простого HTTP-парсинга, в частности динамическими таблицами и всплывающими окнами;

– поддержка распределенного исполнения через Selenium Grid и облачные сервисы, такие как Selenium Grid Hub, BrowserStack;

– возможность интеграции с фреймворками для тестирования, логирования и отчётности.

2.1.4 Лингвистическое обеспечение

При разработке модуля агрегации использовался язык программирования python. Его преимущества заключаются в следующем:

– синтаксис python позволяет снижать когнитивную нагрузку разработчиков и ускоряет написание и сопровождение кода;

– широкий набор специализированных библиотек;

– параллелизм и масштабируемость;

– наличие HTTP-клиентов, что обеспечивает надежный обмен данными с внешними системами в режиме реального времени.

2.1.5 Алгоритмы сбора и обработки данных

Для каждого из трех источников, а именно NVD, БДУ ФСТЭК России, huntr.com, реализованы самостоятельные компоненты сбора данных, учитывающие специфику доступа, формат данных.

Сбор данных из NVD осуществляется через REST-интерфейс. Влияние на выборку задается двумя параметрами: keywordSearch, позволяющим фильтровать уязвимости по описанию и cpeName, указывающим конкретный продукт или компонент по формату CPE.

В цикле по страницам функция формирует параметр startIndex, определяющий смещение от начала общего списка, и resultPerPage, что позволяет получать максимально возможный объем записей за один запрос. Пошагово каждый HTTP-запрос осуществляется методом GET с передачей сформированного словаря параметров. Если сервер возвращает код, отличный от 200, выполнение цикла прерывается, что предотвращает бесконечные попытки при недоступности сервиса.

После успешного получения JSON-документа система начинает поэтапный анализ каждой страницы ответа, сохраняя при этом целостность и согласованность информации. Первым шагом является углубление в структуру, вплоть до массива «vulnerabilities», содержащего информацию о каждой уязвимости. Для каждой такой записи система выделяет область, ключевым элементом которой является «cve». Внутри этого ключа размещается детализированная информация: уникальный идентификатор уязвимости, множественные описания на разных языках и дополнительные атрибуты. Значение поля «cve.id», извлеченное напрямую из JSON, является идентификатором для последующей работы с данными. Текстовое описание извлекается из первого элемента массива «cve.descriptions», обеспечивая получение англоязычной версии описания уязвимости по умолчанию. Если в ответе отсутствуют необходимые ключи, алгоритм автоматически заполнит соответствующие поля заранее заданными значениями по умолчанию. Такой подход гарантирует унифицированную структуру как входных, так и выходных данных, исключая возможность нарушения целостности результирующего набора данных при отсутствии необязательных элементов.

В заключение, на основе собранных и унифицированных значений для каждой уязвимости формируется единый словарь.

Комбинируя в каждой записи уникальный идентификатор, развернутое описание, привязку к категории CWE, унифицированную оценку по стандарту CVSS и другие атрибуты, получается структурированную модель. Блок-схема на рисунке 3, иллюстрирующая этапы извлечения и предобработки данных из NVD, наглядно показывает, как исходные JSON-записи преобразуются в готовый к использованию унифицированный набор данных.



Рисунок 3 – Блок схема алгоритма сбора и обработки данных из NVD

Извлечение сведений из БДУ ФСТЭК России, происходит в два этапа. Первый этап базируется на использовании модуля HTTP-запросов requests. В начале процедуры формируется GET запрос к URL адресу, по которому можно получить XLSX файл с уязвимостями. После получения подтверждения об успешности

загрузки бинарные фрагменты последовательно записываются в локальный файл в двоичном режиме. Второй этап направлен на извлечение структурированных сведений из скачанного XLSX файла и приведение их в единой модели данных, пригодных для обработки и визуализации. Поскольку в данном файле хранятся все уязвимости, независимо от того относятся они к средствам машинного обучения или нет, для последующей обработки необходимо обработать данный файл, а именно отфильтровать только те уязвимости, которые относятся к средствам машинного обучения Для этого с помощью регулярных выражений ищутся строки связанные с данными средствами, в частности их названия. После этого применяется библиотека pandas, позволяющая считывать табличные листы с явным указанием типа данных, что исключает потерю информации при неявном преобразовании числовых и текстовых полей. При загрузки содержимого в DataFrame обход по строкам реализован через метод iterrows, позволяющий получить доступ к каждому элементу таблицы в виде последовательности меток и значений. Первые столбцы обрабатываются как идентификатор уязвимости и её словесное описание, при этом пустые или некорректно закодированные ячейки трактуются как отсутствующие, что позволяет избежать возникновения исключений и сохраняет однородность выходных данных. Особое внимание уделено извлечению оценки критичности по стандарту CVSS версии 3. Соответствующий вектор извлекается из заранее определенной колонки, а числовая составляющая – путем применения регулярного выражения, способного обнаружить число с десятичным разделителем как запятую, так и точку. Итогом обработки служит формирование однородного списка словарей, включающего для каждой уязвимости единую совокупность полей: идентификатор, описание, CWE, вектор CVSS версии 3, числовую оценку CVSS версии 3, и такие же поля для CVSS версии 4. Блок-схема сбора и обработки данных из БДУ ФСТЭК России представлена на рисунке 4.

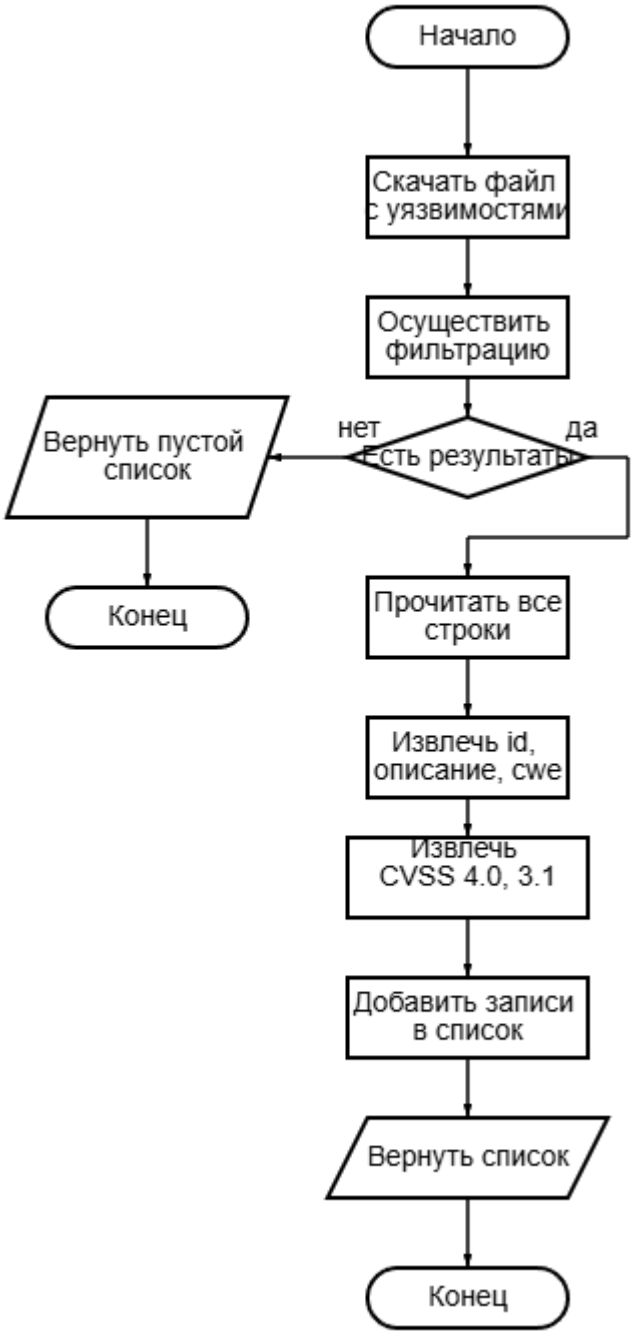


Рисунок 4 – Блок схема алгоритма сбора и обработки данных из БДУ ФСТЭК России

Для извлечения сведений из платформы huntr.com был разработан алгоритм, обеспечивающий автоматизированный сбор сведений о CVE с динамически загружаемой страницы. Первоначально инициируется конфигурация браузера Google Chrome через Selenium WebDriver. Применение опции «–disable-gpu» позволяет выполнять скрипт в средах без графической подсистемы. После

настройки окружения WebDriver мгновенно направляется к URL раздела «Hackivity», где публикуются самые свежие отчёты о найденных уязвимостях. Поскольку контент на странице подгружается динамически при нажатии на кнопку «Show more», алгоритм реализует циклическую задержку, обеспечивая стабильное ожидание завершения запросов. Метод поиска элемента по идентификатору позволяет локализовать корневой узел таблицы с данными, после чего из всех вложенных элементов извлекаются необходимые значения. Для предотвращения утечек ресурсов и гарантированной корректности завершения сессии в модуле явно вызываются методы прекращения работы драйвера и освобождения фоновых процессов. Блок-схема алгоритма представлена на рисунке 5.

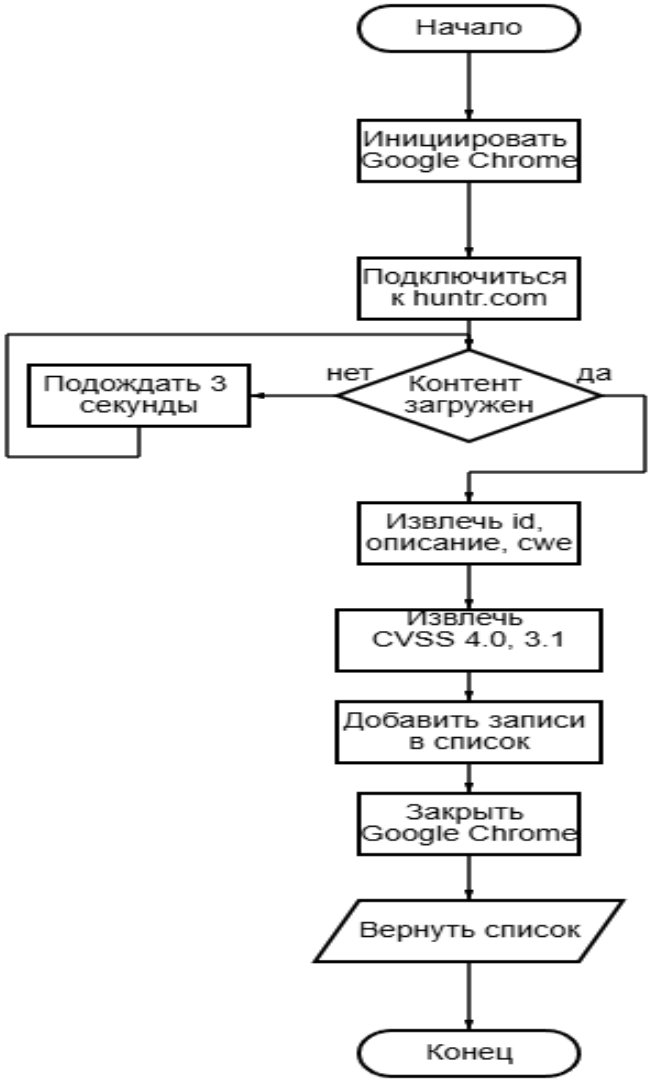


Рисунок 5 – Блок-схема алгоритма сбора и обработки данных с huntr.com

В основе функционирования компонента получения оценки EPSS лежит определение точки взаимодействия с внешним сервисом. Эта точка представлена REST-API платформы EPSS. URL данного интерфейса служит начальной точкой при формировании HTTP-запроса, предназначенного для получения актуальных значений вероятности эксплуатации уязвимости, идентифицируемой CVE. При выполнении функции, отвечающей за всю логику процесса, обмен данными осуществляется через стандартный GET-запрос. На стадии формирования запроса в его заголовки и параметры подставляется необходимый идентификатор CVE, после чего запрос отправляется на сервер EPSS.

Следующий шаг алгоритма посвящен проверке ответа – валидации HTTP-кода состояния на предмет выявления ошибок соединения или некорректного формирования запроса. В случае успешного получения данных происходит парсинг JSON-документа, из которого извлекается численная оценка EPSS из соответствующего поля. Эта оценка отражает текущую вероятность эксплуатации уязвимости.

В ситуациях, когда на любом этапе обмена данными – будь то проблемы с сетью, некорректный ответ сервера или ошибки при десериализации JSON – возникает исключительная ситуация, функция возвращает предварительно установленное значение «0». Такой подход обеспечивает высокую надежность модуля. В случае отсутствия достоверной информации о риске эксплуатации уязвимости, внешние компоненты системы получают однозначный индикатор "недоступности данных" и могут продолжать работу, не прерывая общий процесс анализа. Блок-схема алгоритма, изображенная на рисунке 6, представляет последовательность операций – от инициализации REST-точки до обработки результатов и обработки ошибок.



Рисунок 6 – Блок схема алгоритма получения значения EPSS

На рисунке 7 представлен разработанный графический интерфейс для агрегирования данных об уязвимостях.



Рисунок 7 – Интерфейс модуля агрегирования

В представленном интерфейсе предусмотрена возможность тонкой настройки параметров поиска уязвимостей. В частности, с помощью поля «CPE» (Common Platform Enumeration) задаётся унифицированное наименование программного продукта в соответствии со структурированной схемой NIST [39]. Данный параметр актуален исключительно для базы NVD, поскольку в других источниках, включая БДУ ФСТЭК России, аналогичного признака не предусмотрено. Применение фильтрации по CPE обеспечивает высокую точность выделения лишь тех записей, которые непосредственно относятся к исследуемому программному обеспечению, например, к фреймворкам и библиотекам машинного обучения.

Поиск по ключевым словам реализован посредством регулярных выражений и строгого совпадения строк, что позволяет эффективно обнаруживать упоминания целевых модулей или компонентов даже при вариативности синтаксиса их наименований. При этом использование гибридного режима поиска – когда запрос одновременно обращается к NVD и к БДУ ФСТЭК расширяет покрытие анализа и компенсирует отсутствие CPE-метаданных в отечественном реестре.

Кроме того, предусмотрен режим «Поиск по умолчанию», который автоматически агрегирует запросы на основе заранее сформированного конфигурационного файла. Этот файл содержит как список значений CPE, так и набор ключевых слов, что позволяет запускать повторяемые и воспроизводимые

процедуры сканирования без необходимости ручного ввода каждого параметра запроса.

2.2 Автоматизация оценки рисков атак на средства машинного обучения

Система автоматизированной оценки рисков дает возможность повысить скорость анализа больших объемов данных, что становится важным при наличии сотен и тысяч уязвимостей. При увеличении объема данных ручной метод, заключающийся в оценке каждой уязвимости путем сбора данных и оценки риска, теряет свою эффективность. Для устранения этих проблем был разработан модуль оценки риска, способный принимать на вход множество уязвимостей и выдавать соответствующие им числовые показатели риска. Центральным элементом является компонент оценки риска, взаимодействующий с системой агрегации данных. Задачи решаемые модулем:

– подготовка и нормализация данных: приведение метрик к единой шкале;

– расчет критичности уязвимостей;

– вычисление значения риска.

Работа модуля начинается с подготовки и нормализации данных. Полученные в результате агрегации данные приводятся к единой шкале, так как оценки калькуляторов находятся в разных диапазонах и для корректной работы необходимо произвести нормализацию данных оценок. Нормализованная оценка калькулятора вычисляется по формуле (1).

$$s_{norm} = \frac{s - s_{min}}{s_{max} - s_{min}}, \quad (1)$$

где s_{norm} – нормализованная оценка калькулятора;

s – оценка калькулятора;

s_{max} – максимальная возможная оценка калькулятора;

s_{min} – минимальная возможная оценка калькулятора.

Для реализации метода среднеквадратического значения необходимо рассчитать среднее значение [40]. Среднеквадратическая оценка по каждой уязвимости рассчитывается по формуле (2):

$$s_{final} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} s_{norm,i}^2}, \quad (2)$$

где s_{final} – среднеквадратическая оценка;

$s_{norm,i}$ – нормализованная оценка калькулятора;

n – количество калькуляторов.

Исходя из этой оценки можно получить вероятность эксплуатации уязвимости [41] по формуле (3).

$$p_i = \frac{s_{final,i}}{\sum_{i=1}^{n} s_{final,i}}, \quad (3)$$

где p_i – вероятность эксплуатации уязвимости;

$s_{final,i}$ – нормализованная оценка критичности уязвимости;

i – индекс уязвимости;

n – количество уязвимостей.

Для определения значения ущерба необходимо рассмотреть вектор CVSS. А именно показатели влияния на конфиденциальность, целостность, доступность. Количественной оценкой данных параметров являются значения {0, 0.2, 0.5} для N, L, H соответственно. Используя формулу (4) можно рассчитать значение ущерба.

$$u = 1 - (1 - u_K) \cdot (1 - u_{Ц}) \cdot (1 - u_{Д}), \quad (4)$$

где u – значение ущерба;

u_K – коэффициент, характеризующий влияние на конфиденциальность;

$u_{Ц}$ – коэффициент, характеризующий влияние на целостность;

$u_Д$ – коэффициент, характеризующий влияние на доступность.

После нахождения значений вероятности и ущерба, по формуле (5) возможно найти значение риска.

$$risk = p \cdot u, \quad (5)$$

где $risk$ – значение риска;

p – значение вероятности эксплуатации уязвимости;

u – значение ущерба.

2.2.1 Структура данных

Основным элементом системы является реляционная база данных, построенная на легкой и надежной СУБД SQLite. Каждая запись в ней описывает отдельную уязвимость, идентифицировали по CVE-идентификатору. Помимо этого, в схему включены поля, отражающие классификацию уязвимости по CWE, текстовое описание, привязку к конкретной технике из фреймворка MITRE ATLAS, а также численные показатели по самым распространённым шкалам оценки. CVSS версий 3.1 и 4.0. Вероятность эксплуатации из EPSS, AIVSS, значение влияния на конфиденциальность, целостность и доступность и показатели наличия и зрелости публичных эксплойтов. Такая унифицированная и взаимосвязанная структура данных обеспечивает возможность гибкой фильтрации, отбора и нормализации информации, а также облегчает дальнейшее расширение схемы при появлении новых источников метрик.

2.2.2 Технологическое обеспечение

Выбор технологического стека обусловлен необходимостью высокой производительности, простоты развертывания и масштабируемости. В качестве основы для API-слоя был избран FastAPI – современный асинхронный Python-

фреймворк, позволяющий обрабатывать большое число запросов с минимальными задержками. Взаимодействие с базой данных организовано через ORM SQLAlchemy, что позволяет обращаться к данным на уровне объектов, не теряя в скорости выполнения запросов благодаря использованию SQLite. Модели запросов и ответов строго типизированы с помощью Pydantic, что гарантирует корректность входных данных и облегчает отладку. Встроенные модули стандартной библиотеки Python, такие как math для вычислений и itertools для генерации комбинаций путей атак, дополняют картину лёгкой и понятной структуры кода, при этом оставляя пространство для масштабирования и внедрения новых компонентов.

2.2.3 Функциональные библиотеки

При реализации ключевых функциональных блоков системы использовался набор как внешних, так и стандартных библиотек.

FastAPI – современный и быстрый фреймворк для создания API с помощью python. Особенности данного фреймворка:

– использует асинхронную модель, что позволяет обрабатывать сотни запросов в секунду;

– из аннотаций типов автоматически формируются интерактивные OpenAPI страницы;

– декларативное описание маршрутов делает код лаконичным и легко читаемым;

– встроенный механизм «зависимостей» упрощает управление сервисами.

Uvicorn – высокопроизводительный асинхронный сервер, разработанный для веб-приложений на базе python. Используется для запуска приложений FastAPI. Выделяется своей скоростью и нативной поддержкой асинхронности. Особенности данного сервера:

– при разработке можно автоматически перезагружать приложение при изменении исходников;

– легко масштабируется в связке с балансировщиками нагрузки.

SQLAlchemy – python библиотека для работы с базами, использующими объектно-реляционное отображение. Упрощает определение схемы и формирование оптимальных SQL-запросов к SQLite. Можно выделить следующие особенности:

– позволяет писать как чистые SQL-запросы, так и программные;

– управляемое изменение схемы базы данных без потери данных.

Pydantic – библиотека для проверки и сериализации данных, основанная на аннотациях типов. Позволяет определять модели данных с помощью классов, а затем использовать эти модели для преобразования данных. Строго проверяет структуру сообщений, что на этапе разработки значительно уменьшает число ошибок. Выделенные особенности:

– проверяет форматы и типы данных при создании моделей и производит автоматическую конвертацию входящих значений;

– написана на Cython, поэтому значительно быстрее аналогов на чистом python.

Для математических вычислений и статистической обработки используются функции из модуля math, особенностями которого являются:

– вычисление базовых математических функций;

– функции реализованы на C и оптимизированы для быстрого выполнения.

Для перебора всевозможных сценариев атак используется инструмент itertools. Особенности инструмента:

– комбинаторные генераторы, которые позволяют легко перебирать всевозможные варианты;

– все итераторы «ленивые», то есть элементы создаются по требованию, без хранения в памяти всего списка.

2.2.4 Лингвистическое обеспечение

Работа с текстовыми описаниями уязвимостей и названиями техник требует единого подхода к кодировке и нормализации. В системе используется кодировка UTF-8, что позволяет корректно хранить как русскоязычные, так и англоязычные

описания. Для обеспечения сопоставимости по техникам атаки интегрирован справочник MITRE ATLAS: все текстовые метки автоматически сверяются с официальным набором.

2.2.5 Алгоритм оценки риска и составления сценариев

Процесс вычисления итогового значения риска включает два взаимосвязанных этапа.

Первый этап заключается в нормализации оценок множества калькуляторов. После получения их из базы данных, с помощью формулы (1) происходит их нормализация. С использованием формул (2-3) рассчитывается значение вероятности каждой уязвимости. С использованием формулы (4) рассчитывается значение ущерба. По формуле (5) находится значение риска. Следующим этапом становится построение сценариев атак.

Из базы данных выбираются все техники, задействованные в сценарии, и для каждой техники формируется список уязвимостей с оценкой вероятности эксплуатации.

При построении сценариев используется лучевой поиск, который представляет собой эвристическую стратегию поиска с ограничением ширины «луча». Это позволяет находить приемлемые решения без экспоненциального роста вычислительных затрат. Лучевой поиск хранит лишь k наиболее перспективных состояний, где k – заданная ширина «луча». Такой подход позволяет балансировать между качеством результата и вычислительными затратами. Преимущества данного варианта:

– ограниченная память: хранится не все разветвления, а только k лучших;

– ускорение поиска: за счёт отсечения менее перспективных ветвей существенно снижается число обрабатываемых узлов;

– гибкость: можно регулировать качество и используемые ресурсы, меняя размер «луча».

Однако данный вариант не лишен недостатков. Недостатки данного метода:

– отсутствие полноты: лучевой поиск может пропустить оптимальное решение, если нужная ветвь не попадет в топ-k на каком-то уровне;

– алгоритм может остановиться на локально хороших, но глобально не оптимальных решениях;

– требует тонкой настройки: выбор k зачастую решает всё. Слишком маленькое значение k – риск пропуска, слишком большое – теряется преимущество экономии ресурсов.

Другим возможным алгоритмом при построении сценариев может быть поиск в ширину. Преимущества данного алгоритма:

– полнота: если существует решение на конечной глубине, алгоритм обязательно его обнаружит;

– просто реализации: основан на очереди, что легко понимается;

– детерминированность: порядок обхода строго задается уровнями, результаты воспроизводимы.

Недостатки:

– экспоненциальная память: хранит все вершины текущего уровня;

– высокая вычислительная сложность.

Таким образом, получается множество сценариев с вычисленным значением риска. По такой же схеме происходит поиск сценария с наибольшей вероятностью и наибольшим ущербом.

На рисунке 8 представлена страница из разработанного программного обеспечения.

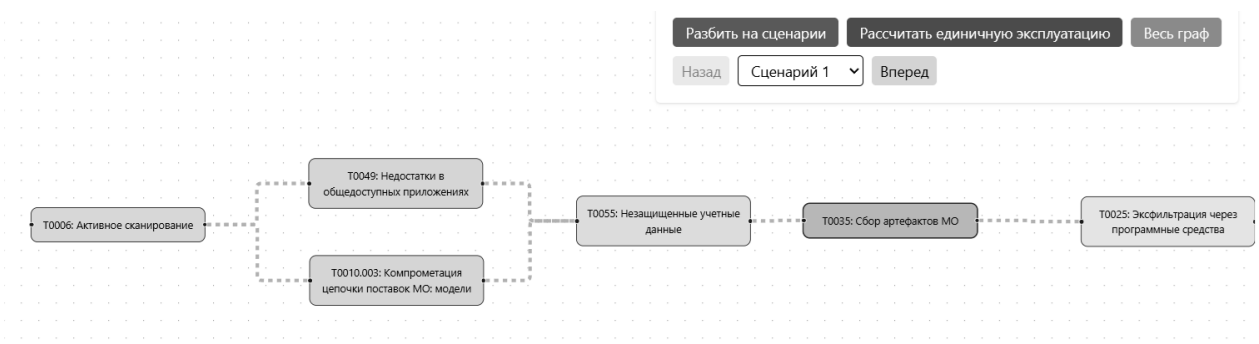


Рисунок 8 – Страница с построенным сценарием

На данной странице изображен сценарий. Ключевыми элементами управления являются кнопки «Вперед» и «Назад» которые из множества сценариев, поданных на вход, производят построение сценария. Кнопка «Рассчитать единичную эксплуатацию» запускает работу модуля оценки риска. На рисунке 9 представлено окно, которое появляется при нажатии на блок с техникой.

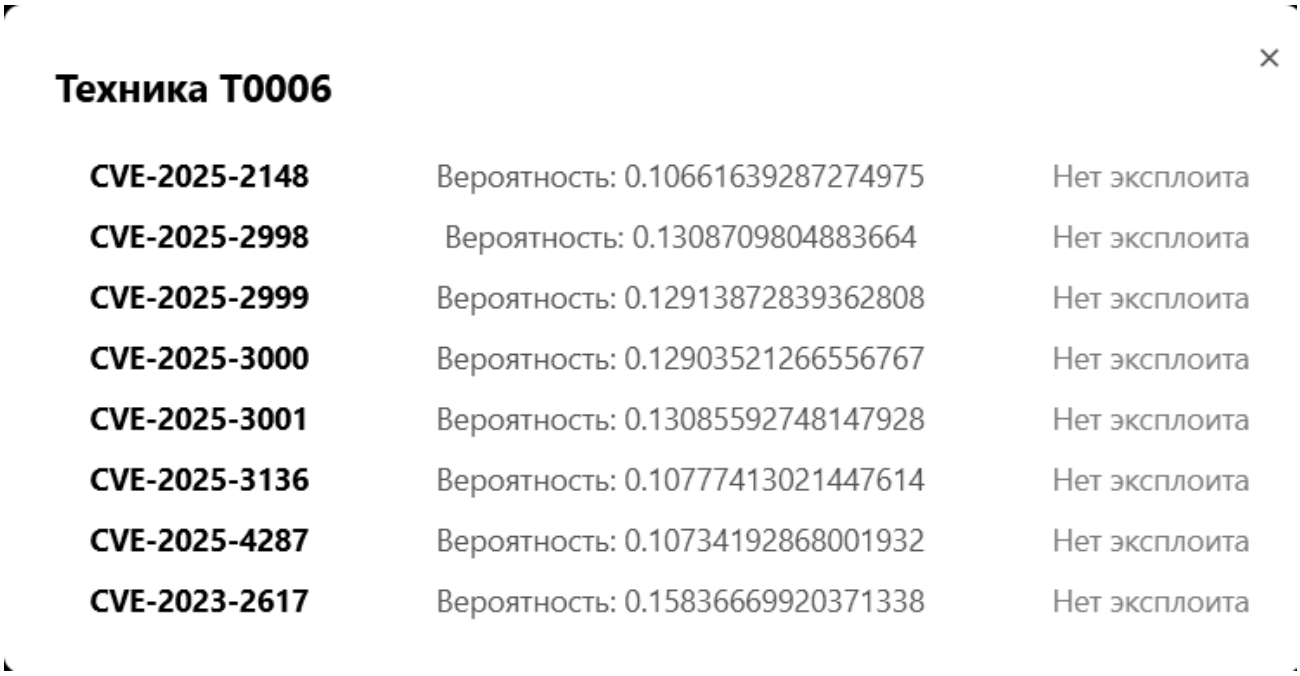


Рисунок 9 – Окно с уязвимостями техники

При нажатии на технику отправляется запрос к базе данных и осуществляется поиск всех уязвимостей, ассоциированных с ней. После получения этих данных производится расчет вероятности эксплуатации каждой уязвимости по формулам (1-3).

На рисунке 10 представлена страница после нажатия на кнопку «Рассчитать единичную эксплуатацию». Здесь предполагается, что одна техника эксплуатирует только одну уязвимость и на основе этого происходит поиск всех возможных вариантов. Затем по формуле (5) производится оценка риска. Прежде всего, выводятся два сценария. Одним из них является сценарий с наибольшей вероятностью, другим сценарий с наибольшим значением ущерба. Затем выводятся все остальные сценарии.

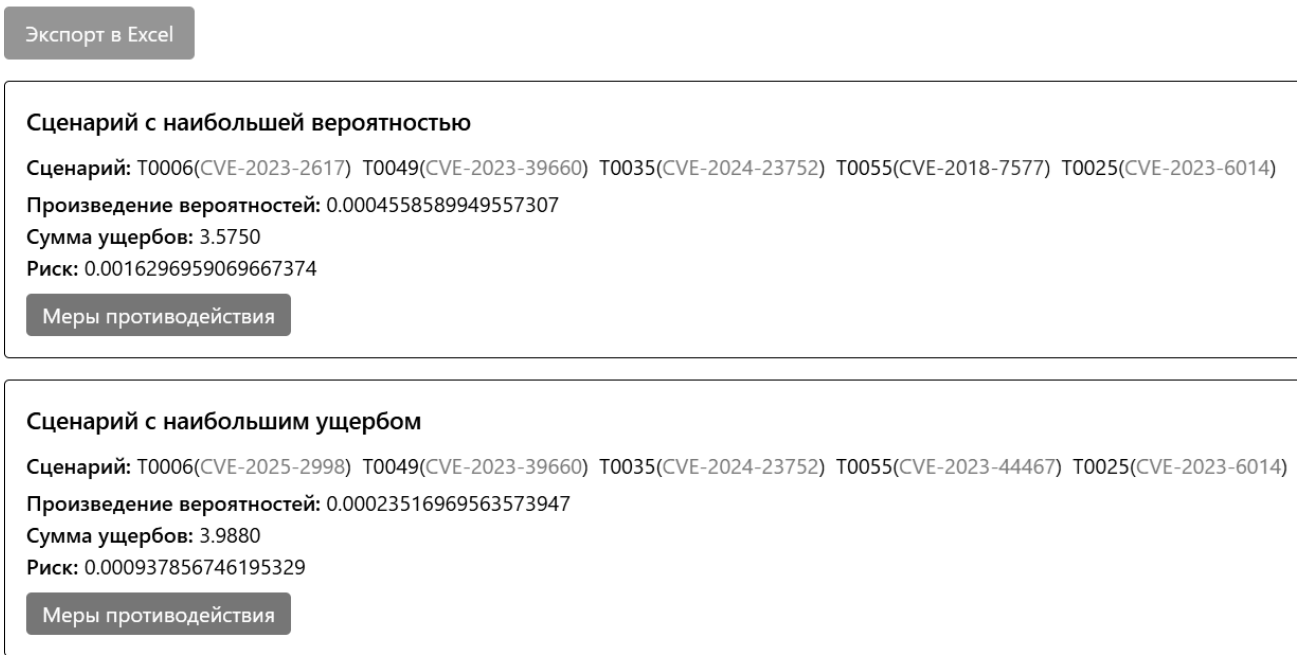


Рисунок 10 – Страница со сценариями

Карточка сценария содержит несколько ключевых полей:

– сценарий, который состоит из последовательности техник и уязвимостей, которые они эксплуатируют;

– произведение вероятностей всех уязвимостей;

– сумму значений ущерба;

– значение риска;

– кнопку «Меры противодействия».

Кнопка «Меры противодействия» инициирует работу модуля по генерации мер противодействия.

2.3 Автоматизация сопоставления уязвимостей с техниками

Система сопоставления уязвимостей с техниками разработана как модуль для автоматического анализа описаний уязвимостей и их классификации по техникам MITRE ATLAS. Целью данного модуля является обеспечение унифицированного интерфейса для анализа любых описаний уязвимостей и автоматического сопоставления их с формальным набором техник MITRE ATLAS. А также

предусмотреть адаптивность – возможность регулярного дообучения на расширяющихся данных, включая случаи нестандартных или редких классов. Задачи, решаемые модулем:

– загрузка и подготовка данных, включающая чтение датасета, предварительную фильтрацию и очистку описаний, формирование единого текстового поля;

– обучение модели;

– загрузка сохраненной модели и токенизатора;

– определение метки техники MITRE ATLAS.

На вход подаётся пара «текстовое описание уязвимости + идентификатор CWE». В основе архитектуры лежит двухэтапный конвейер: сначала выполняется предобработка и векторизация текстовых данных, затем обученная модель предсказывает наиболее вероятную технику MITRE ATLAS. Такой подход обеспечивает гибкость при возникновении новых техник и упрощает расширение функциональности за счёт возможности переобучения модели на обновленном наборе данных.

2.3.1 Структура данных

В качестве исходных данных используется датасет CMAD_V1 [42], содержащий три ключевых поля: текст описания уязвимости, идентификатор CWE, целевой идентификатор техники MITRE ATLAS. Каждое описание предварительно очищается и объединяется с идентификатором CWE, что создаёт единое пространство признаков для модели. Чтобы подготовить данные к обучению, для каждой уникальной техники формируются словари соответствия, обеспечивающие согласованное представление классов в процессе обучения и инференса. Также происходит нормализация текста. Удаляются HTML-теги, ссылки и пунктуация, текст приводится к нижнему регистру и устраняются лишние пробелы. Такое преобразование гарантирует, что семантическое описание уязвимости сохраняется, а шумовые маркеры не влияют на качество обучения модели. Токенизация

осуществляется при помощи AutoTokenizer, специализированного на работу с BERT-подобными архитектурами, что позволяет корректно обрабатывать как технические термины, так и естественно-языковые конструкции.

2.3.2 Технологическое обеспечение

Модуль построен на python с использованием библиотеки transformers от Hugging Face для работы с предобученными языковыми моделями. Обучение модели организовано с помощью API Trainer и TrainingArguments, что упрощает настройку стратегий логирования, оценки и сохранения контрольных точек. Гиперпараметрический поиск проводится через фреймворк Optuna, интегрированный в конвейер обучения, что позволяет автоматически подбирать оптимальные параметры обучения, для максимизации метрики точности.

2.3.3 Функциональные библиотеки

Для предобработки и анализа данных используется pandas и стандартные модули python, такие как re, os. Для разделения выборок и расчета метрик привлекаются инструменты из sklearn, в частности train_test_split, accuracy_score, precision_recall_fscore_support.

Sklearn – это библиотека языка python, данная библиотека предназначена для машинного обучения и распространяемая в виде свободного программного обеспечения. В ее структуру включены разнообразные алгоритмы, охватывающие широкий спектр задач. Сюда относятся методы классификации, регрессии и кластеризации данных, применяемые для решения аналитических задач. Среди них выделяются такие инструменты, как метод опорных векторов, случайный лес, повышение градиента, k-средних и DBSCAN [43-47]. Разработка данной библиотеки предусматривала её полную совместимость и тесное взаимодействие с математическими библиотеками языка Python, такими как NumPy и SciPy. «Train_test_split» позволяет разделить массивы на произвольные обучающие и

тестовые подмножества. Это быстрая утилита, которая совмещает проверку входных данных и применение к вводу в одном вызове для разбиение данных в одну строку кода. Accuracy_score позволяет оценить правильность классификации. В многомаркерной классификации эта функция вычисляет точность подмножества: набор меток, предсказанных для примера, должен в точности соответствовать набору меток в истинном множестве меток. Precision_recall_fsore_support вычисляет для каждого класса следующие метрики [48]:

– точность – отношение числа верно предсказанных образцов данного класса к сумме верно предсказанных и ошибочно отнесенных к данному классу. Точность показывает, какую долю предсказаний положительного класса составляют действительно правильные;

– полнота – отношение числа верно предсказанных образцов данного класса к сумме верно предсказанных и числа образцов, ошибочно не распознанных. Полнота отражает способность классификатора находить все образцы данного класса

– F-мера – взвешенное гармоническое среднее точности и полноты;

– поддержка – число образцов каждого класса в истинных метках.

Обучение реализовано на базе PyTorch [49], фреймворка машинного обучения для языка python с открытым исходным кодом, созданного на базе Torch. Используется для решения различных задач, таких как компьютерное зрение, обработка естественного языка. PyTorch предоставляет две основные высокоуровневые модели:

– тензорные вычисления с развитой поддержкой ускорения на GPU;

– глубокие нейронные сети на базе системы autodiff.

Библиотека transformers предоставляет тысячи предварительно обученных моделей для выполнения различных задач, таких как текст, зрение и аудио. Transformers предоставляет API для быстрой загрузки и использования предварительно обученных моделей, их тонкой настройки на собственных датасетах и последующего взаимодействия с ними. Transformers опирается на три самые популярные библиотеки глубокого обучения - Jax, PyTorch и TensorFlow - и легко

интегрируется между ними. Это позволяет легко обучать модели с помощью одной из них, а затем загружать их для выводов с помощью другой.

Optuna – это программная среда автоматической оптимизации гиперпараметров, специально разработанная для машинного обучения. Она имеет императивный пользовательский API, благодаря чему обладает высокой модульностью. Отличительными особенностями являются:

– легкая, универсальная и независимая от платформы архитектура;

– поисковые пространства python;

– эффективные алгоритмы оптимизации;

– простое распараллеливание;

– быстрая визуализация.

2.3.4 Лингвистическое обеспечение

Подобно тому, как это было реализовано в системе для сбора и анализа информации об уязвимостях в средствах машинного обучения, текущая система также разработана на языке python. При создании использован широкий набор его функциональных библиотек – от стандартных модулей для работы с данными и сетевым взаимодействием до специализированных пакетов для научных вычислений и машинного обучения. Он является одним из самых популярных и востребованных языков в машинном обучении [50-52]. Такой выбор набора технологий позволяет обеспечить высокую гибкость, масштабируемость и читаемость кода, а также значительно ускоряет процесс разработки и интеграции новых компонентов.

2.3.5. Алгоритм сопоставления уязвимостей с техниками

В самом начале работы над моделью ключевое значение имеет этап её обучения, который стартует с подготовленного и заранее очищенного набора данных. Этот датасет формируется таким образом, чтобы наиболее полно представлять все целевые классы, охватывая их многообразие. После сбора образцы

подвергаются балансировке, предполагающей установление идентичного количества примеров для каждого класса. На этапе предобработки также устраняются различные помехи и искажения. Например, удаляются выбросы, которые могут появиться из-за погрешностей сенсоров или непреднамеренных ошибок. Кроме того, корректируются систематические артефакты, вызванные непоследовательностью в исходных данных.

Непосредственно во время обучения осуществляется итеративная настройка параметров модели. В каждой итерации вычисляется величина функции потерь на обучающем наборе данных. Сопоставляя значение ошибки и величину шага обновления, алгоритм постепенно уменьшает расхождения между предсказанными значениями и эталонными.

На рисунке 11 представлен график зависимости функции потерь от шага обучения. На начальных этапах обучения происходит стремительное уменьшение ошибки, свидетельствующее о быстром росте предсказательной способности. По мере приближения к завершению обучения кривая потерь начинает выравниваться, указывая на достижение оптимальных параметров.

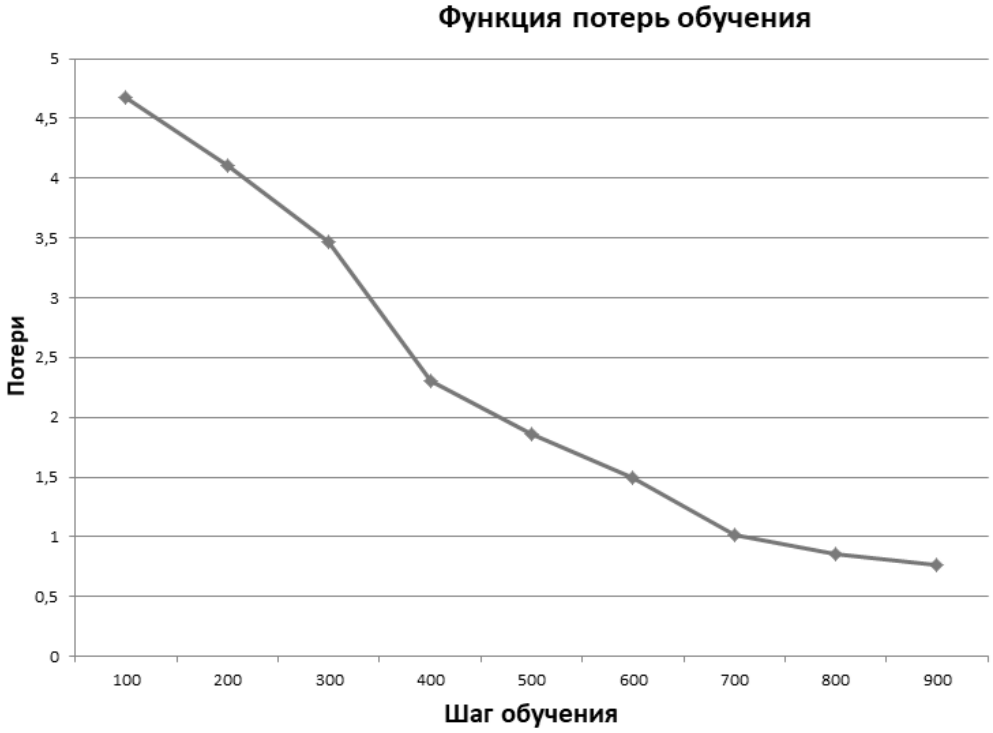


Рисунок 11 – График зависимости функции потерь обучения от шага обучения

Другой важной функцией является функция потерь на валидации. Это та же функция потерь, что и при обучении, но данная функция вычисляется на отдельном наборе данных. Во время обучения происходит минимизация функции потерь обучения. Это показывает, как модель запомнила обучающую выборку. Значение функции потерь на валидации показывает, как модель будет вести себя на данных, которые она раньше не видела. График функции потерь на валидации представлен на рисунке 12.

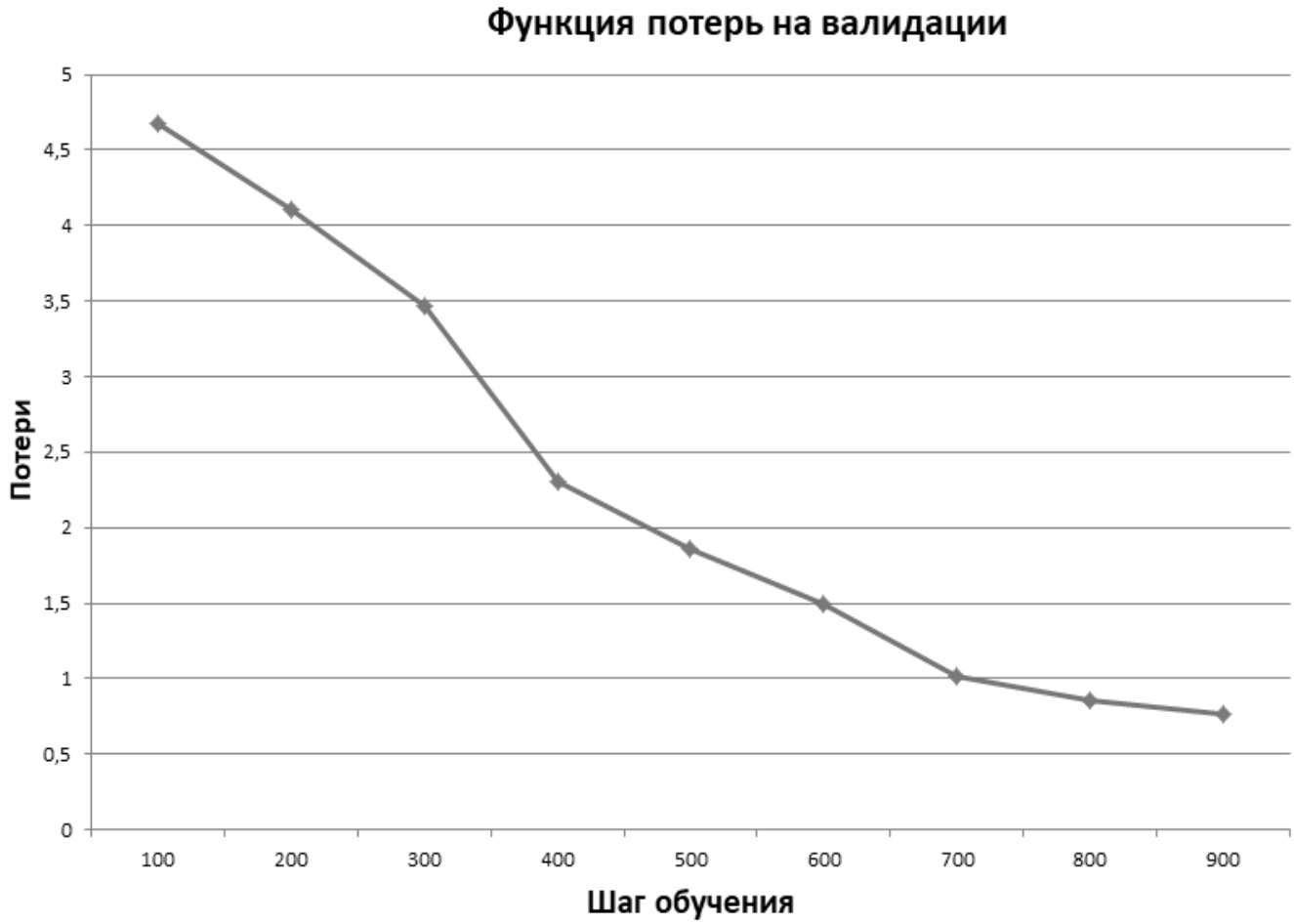


Рисунок 12 – График функции потерь на валидации

На представленной кривой между 100 и 300 шагом значение функции потерь падает с примерно 4.2 до 2.8, что показывает, что модель быстро осваивает базовые закономерности. После 300-400 шага кривая плавно выравнивается. Это классический эффект убывающей отдачи. На каждом новом шаге модель учится всё медленнее. Значения не растут ни на одном участке, а значит модель не «заучивает»

шум из тренировочной выборки. К середине-концу обучения кривая почти выпрямилась, дальнейшее снижение очень незначительно.

Метрика точности отражает долю корректных предсказаний модели на наборах данных. График изменения точности позволяет наглядно отслеживать, когда модель начинает учиться и когда процесс обучения замедляется. График точности модели представлен на рисунке 13.

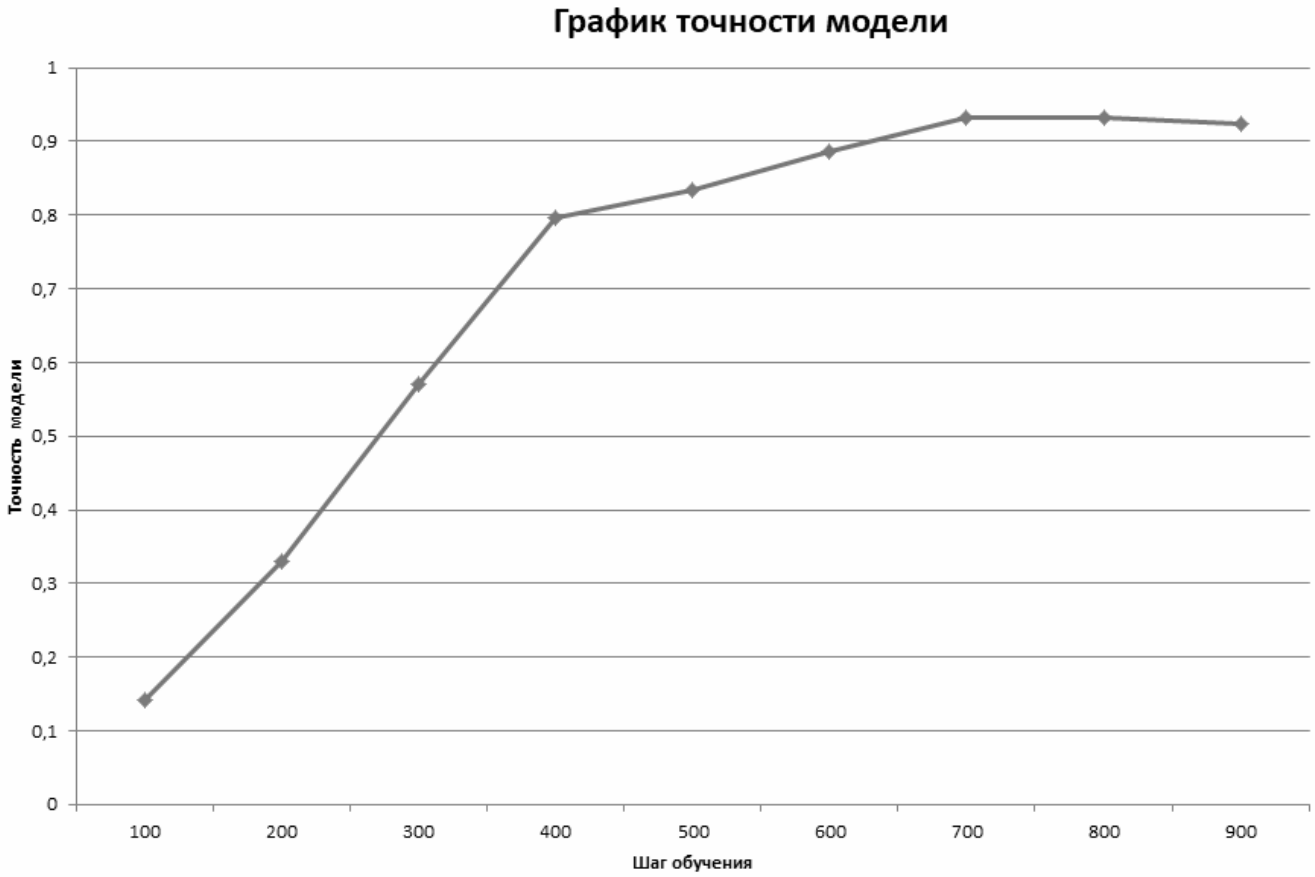


Рисунок 13 – График точности модели

В первые 100-400 шагов модель демонстрирует впечатляющий рост точности, с 0.14 до порядка 0.80. Это обычно для начального этапа. По мере продвижения к 700 шагу прирост замедляется, точность постепенно поднимается с 0.8 до примерно 0.93, что говорит о том, что легкие паттерны уже учтены, и последующая оптимизация требует тонкой настройки. После 700 шагов кривая почти выравнивается, а к 900 шагу наблюдается небольшое снижение до 0.92, что может

служить сигналом о начале переобучения или влиянии шумовых особенностей выборки.

Данная динамика демонстрирует оптимальный интервал обучения. Продолжение обучения после 700-800 шага приносит минимальную выгоду и повышает риск переобучения.

В ходе инференса модуль последовательно выполняет несколько ключевых этапов. Сначала на загружаются два основных компонента, сохранённые при обучении: сама модель и соответствующий ей токенизатор. В качестве входных данных система принимает текст, который предварительно формируется объединением двух частей исходного CWE и текстового описания конкретной уязвимости. Получившийся фрагмент подвергается токенизации и затем преобразуется в тензоры заранее заданной фиксированной длины: более короткие последовательности дополняются при необходимости специальными «пустыми» токенами, а слишком длинные обрезаются до нужного размера.

Далее модель переводится в режим оценки, при котором отключаются все механизмы, связанные с обучением. На этом этапе модель обрабатывает входные тензоры и для каждого из возможных классов-меток вычисляет не нормированные вероятности. Чтобы получить окончательный прогноз, к вектору логитов применяется операция argmax: определяется индекс класса с наибольшим значением. Последним шагом происходит обратное преобразование, числовой идентификатор выбранной техники сопоставляется с соответствующим строковым названием. В результате модуль выдаёт пользователю метку уязвимости. Общая схема работы описанных компонентов наглядно представлена на рисунке 14.



Рисунок 14 - Блок схема работы модуля сопоставления уязвимостей с техникой

2.4 Алгоритмизация генерации мер противодействия

В современных системах информационной безопасности критически важно не только находить и анализировать уязвимости, но и мгновенно формировать соответствующий набор мер защиты. Для этого используется автоматизированная

подсистема генерации мер противодействия [53]. Она получает на вход детальное описание обнаруженной уязвимости. Кроме того, здесь осуществляется интеграция с системой сопоставления уязвимостей с техникой ATLAS. Это позволяет подсистеме, основываясь на описаниях уязвимостей и техниках MITRE ATLAS, формировать структурированный список как технических, так и организационных мер, разработанных с учетом выявленной уязвимости. Подобный подход к регламентации позволяет оперативно реагировать на текущие угрозы, а также активно устранять потенциальные риски [54-55].

2.4.1 Структура данных

В качестве исходного материала для обучения и работы генератора мер используется таблица, где каждая строка соответствует одной уязвимости. Основным полем является текстовое описание уязвимости, дополненное списком применяемой техники. К этому к этой основе прикреплены заранее подготовленные категории рекомендаций: базовые организационные и технические меры, компенсирующие меры. Данные хранятся в формате CSV и загружаются в удобную для обработки структуру DataFrame, что позволяет легко проверять соответствие и полноту заполнения каждой категории мер, а также расширять их список при необходимости.

2.4.2 Технологическое обеспечение

Основой генерации мер служит модель трансформера, реализованная на базе фреймворка PyTorch и библиотеки Transformers. Процесс обучения и развертывания модели автоматизирован в скриптах, которые последовательно обрабатывают исходные данные, выполняют тренировку и сохраняют итоговое состояние модели для быстрого запуска в продуктивной среде. Логика генерации вынесена в отдельный модуль, чтобы при необходимости её можно было подключить к любому сервису или интерфейсу.

2.4.3 Функциональные библиотеки

В работе генератора задействованы следующие ключевые модули и библиотеки:

Библиотека Transformers для работы с предобученными моделями и соответствующими токенайзерами. Основные возможности и особенности:

– мгновенный доступ к сотням моделей;

– единый API для задач генерации текста, классификации, заполнения масок, перевода;

– совместимость с accelerate, что упрощает распределенное обучение и использование нескольких GPU;

– готовые тренировочные скрипты.

PyTorch – фреймворк глубокого обучения для построения нейронных сетей. Используется для выполнения этапов обучения и инференса. Основные возможности:

– многомерные массивы с поддержкой вычислений на CPU и GPU;

– автоматическое дифференцирование;

– граф вычислений строится по ходу выполнения, что упрощает отладку.

Благодаря модульному дизайну, при необходимости легко добавить инструменты для мониторинга качества рекомендаций или интеграции с внутренними системами управления уязвимостями.

2.4.4 Лингвистическое обеспечение

Для работы с текстом используется единая кодировка UTF-8, что гарантирует корректное представление как русскоязычных, так и англоязычных записей. Перед обучением все описания и шаблоны рекомендаций приводятся к единому формату: шаблон содержит четко разграниченные разделы («Описание уязвимости», «Базовые меры», «Компенсирующие меры»). Этот единообразный подход позволяет

модели научиться воспроизводить понятную и читаемую структуру в генерируемых рекомендациях.

2.4.5 Алгоритм генерации мер противодействия

Сначала необходимо обучить модель на тренировочном датасете, после чего можно переходить к генерации.

Для модели обученной для генерации мер противодействия также целесообразно построить основные графики и проанализировать их. График функции потерь обучения представлен на рисунке 15.

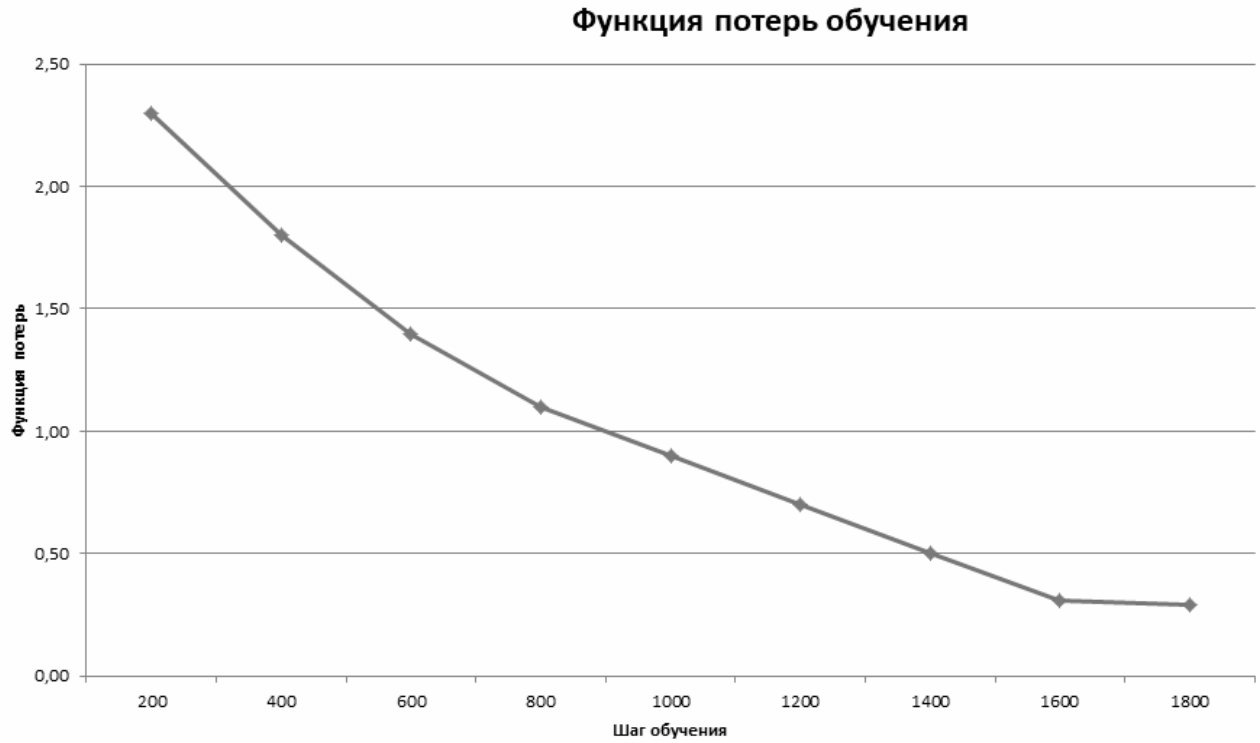


Рисунок 15 – График функции потерь обучения

Значение функции падает без скачков и выбросов. Это говорит о том, что оптимизатор стабильно находит всё более удачные значения параметров. На первых 600-800 шагах значение падает довольно резко. Примерно после 1000 степень уменьшения спадает. Такое поведение характерно для большинства методов

оптимизации со снижением градиентов по мере приближения к минимуму. После 1600 шага модель постепенно выходит на плато.

График функции потерь на валидации представлен на рисунке 16.

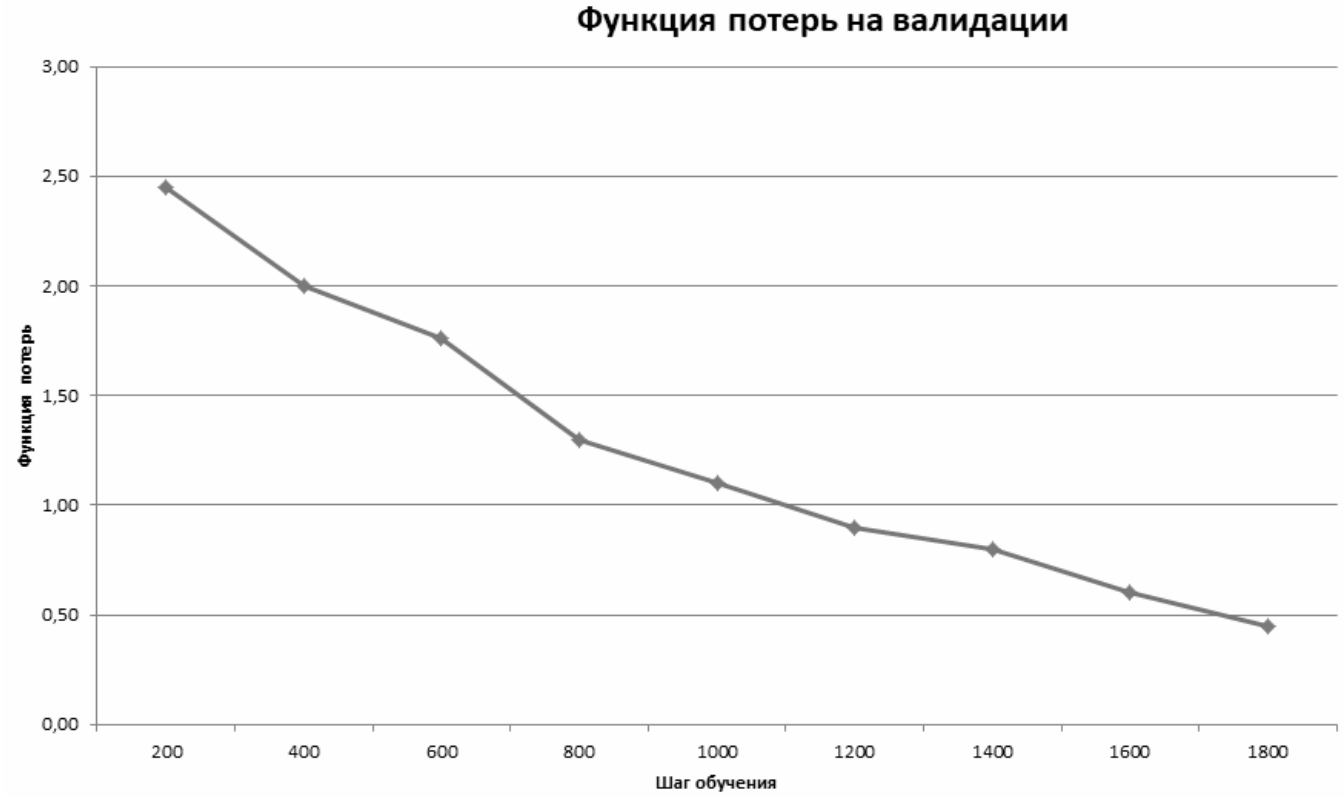


Рисунок 16 – График функции потерь на валидации

Потери на валидационном наборе плавно снижаются без резких скачков и всплесков. Это значит, что модель обобщает данные стабильно. Как и на обучении, на первых 600-800 шагах падение довольно резкое, далее темп замедляется. Функция потерь не отстает от обучения и не начинает расти после определенного шага, что является признаком отсутствия переобучения. Актуальна и в данном случае идея ранней остановки. Если спустя, например 2000-2200 шагов, функция потерь перестанет заметно падать, стоит зафиксировать модель. Таким образом модель учится без признаков переобучения, можно смело увеличить число шагов, либо снизить шаг оптимизатора, что позволит сделать более тонкую настройку. График точности модели представлен на рисунке 17.

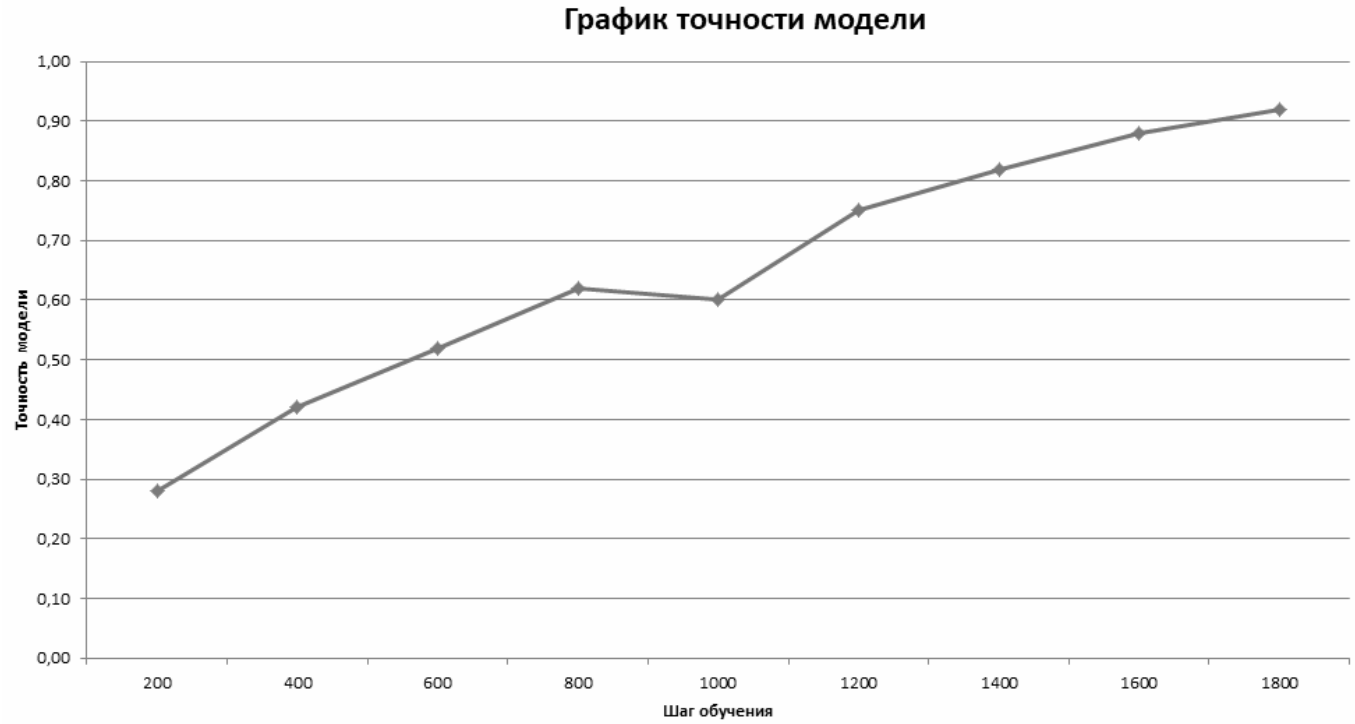


Рисунок 17 – График точности модели

График точности модели коррелирует с кривыми функции потерь. Модель последовательно учится всё лучше распознавать паттерны в данных. Около 1000 шага виден небольшой спад. Такое колебание нередко проявляется из-за стохастики или резкого изменения градиентного шага. После спада идет крупное улучшение. Сессия обучения встала на более спокойный режим, что дало модели возможность перестроиться ближе к глобальному минимуму.

При запуске процесса генерации система последовательно обрабатывает текст описания, сопоставляет его с техникой атаки и передает на вход модели. Модель формирует последовательность рекомендаций, структурированную по заранее заданным категориям, и возвращает готовый отчет, в котором блоки мер расположены в логическом порядке. Результатом работы является связный и подробный набор указаний, который легко интегрировать в отчеты по безопасности или в план мероприятий ответственных команд. На рисунке 18 представлена работа данного модуля. Для данной техники и соответствующей уязвимости генерируются меры противодействия.

T0049 (CVE-2023-39660)

Базовые организационные меры:
Внедрение политики управления уязвимостями, включающей в себя этапы мониторинга (сканирования) и оценки уязвимостей, устранения уязвимостей, назначение ответственных должностных лиц. Установление процедуры еженедельного мониторинга наличия обновлений и установки безопасных и актуальных версий программного обеспечения.
Базовые технические меры:
Обновить платформу управления жизненным циклом машинного обучения Mlflow до актуальной версии 3.1.0. Установка обновления должна происходить только из доверенных источников и лишь после оценки всех сопутствующих рисков
Компенсирующие организационные меры:
Регламент по ограничению доступа к уязвимым компонентам из внешних сетей, в случае возникновения инцидента. Инструкция по проверке и ограничению разрешений файловой системы на рабочих узлах СМО. Методика валидации входных параметров URI при развертывании моделей МО. Использование процедуры изоляции компонентов СМО с уязвимостями. Политика безопасной настройки и эксплуатации платформ МО с веб-интерфейсом.
Компенсирующие технические меры:
Использование межсетевого экрана уровня приложений (WAF) для фильтрации HTTP/HTTPS-запросов Ограничение доступа к уязвимому программному обеспечению с применением белых списков IP-адресов Сегментирование сети для ограничения доступа к уязвимому ПО из других подсетей Использование SIEM-систем для отслеживания попыток эксплуатации уязвимости Использование систем обнаружения и предотвращения вторжений (IDS/IPS) Ограничение доступа к платформе из внешних сетей (Интернет) Ограничение возможности обработки уязвимым ПО запросов, содержащих вредоносные заголовки (например, x-middleware-subrequest) Использование антивирусного ПО с функцией контроля веб-ресурсов Использование замкнутой программной среды Ограничение прав файловой системы и прав пользователей mlflow-сервиса.

Превентивные меры:
1) Ограничение доступа к API логического вывода и журналам принятия решений модели с реализацией минимизации утечки информации о внутренних механизмах RBAC. 2) Использование детекторов аномалий и проверка нетипичных характеристик входных данных перед их вводом в модель. 3) Применение процедуры нормализации, фильтрации и проверки правильности входных данных (конвейер предварительной обработки входных данных) перед вводом в модель. Преобразование входных данных в согласованный формат.
Меры реагирования:
1) Интеграция правил поведенческого анализа в систему мониторинга SIEM, которые фиксируют нестандартную активность модели: массовые ошибки классификации, неожиданное распределение ответов. 2) Реализация механизма уведомления при получении входных данных, характеризующихся высокой степенью отклонения от обучающего распределения (системы мониторинга МО-инференса). 3) Использование средств оркестрации для временного ограничение доступа к модели из внешних интерфейсов до завершения анализа и доработки
Меры ликвидации:
1) Повторное обучение или переподготовка модели на основе расширенного тренировочного набора, включающего фиксированные примеры атак. 2) Проверка СЗИ целостности компонентов логического вывода (исходный код, веса, конфигурации), сверка с эталонными хэш-суммами и цифровыми подписями. 3) Обновление правил в DLP, WAF и IDS/IPS на основании выявленного паттерна обхода. 4) Документирование SIEM-системой инцидента обхода, анализ его сценария, пересмотр модели угроз и корректировка системы реагирования. 5) Использование межсетевого экрана (WAF) или VPN для временного ограничения доступа к модели из внешних интерфейсов до завершения устранения инцидента

Рисунок 18 – Генерация мер противодействия

Построенная архитектура является модульным монолитом. При таком подходе приложение остается единым целым внешне, но внутри четко разделяется на самостоятельные области ответственности – модули. Основными модулями являются модуль агрегирования данных, модуль оценки риска, модуль сопоставления уязвимости с техникой, модуль генерации мер.

Каждый модуль инкапсулирует свою предметную логику, данные и интерфейсы взаимодействия с остальными частями системы, что позволяет разработчикам работать с более мелкими частями кода. При этом внутренние границы между модулями могут быть формализованы через контакты API. Подобная организация способствует более высокой согласованности.

Следовательно, модульный монолит представляет собой оптимальный вариант, сочетающий в себе удобство управления обширными кодовыми базами и экономию

на инфраструктурных затратах, характерные для монолитных приложений, с гибкостью, масштабируемостью и удобством поддержки, присущими микросервисной архитектуре. Корректное выделение модулей, соблюдение принципов инверсии зависимостей, единой ответственности и четких контрактных границ позволяет постепенно развивать систему, двигаясь от монолита к распределенной архитектуре.

Данная схема, содержащая все основные элементы и принципы взаимодействия модулей и источников данных, представлена в Приложении А. Приложение Б содержит фрагменты ключевых алгоритмов.

3 Управление информационной системой

3.1 Руководство пользователя

Данное руководство предназначено для пользователей и администраторов системы, предназначенной для выявления уязвимостей и разработки мер защиты от атак на средства машинного обучения. Основная задача руководства – помочь в освоении интерфейса приложения, научить эффективному использованию его возможностей, а также предоставить решения для типичных проблем, возникающих при его использовании.

Целевая аудитория этого руководства – пользователи, задействованные в анализе рисков и создании защитных мер для предотвращения атак на системы машинного обучения.

Руководство подробно описывает весь цикл работы с системой, начиная с процесса входа в приложение и заканчивая способами устранения возможных неполадок, что обеспечивает всестороннюю поддержку пользователей в выполнении их задач.

Руководство организовано для удобства навигации и использования, разделено на четкие разделы:

– доступ к системе: описание процедуры входа в приложение, включая регистрацию и авторизацию;

– работа с системой: подробная инструкция по основным функциям, таким как оценка рисков и формирование мер защиты;

– типичные проблемы и способы их решения: распространённые ошибки и способы их устранения, представленные в понятных пошаговых инструкциях. Особое внимание уделяется профилактике проблем.

Для облегчения восприятия информации в руководство включены визуальные материалы, такие как скриншоты интерфейса и последовательности действий пользователя. Это снижает вероятность ошибок и ускоряет освоение системы.

3.1.1 Установка и первоначальная настройка

Система может быть развернута локально. Для начала нужно убедиться, что на локальной машине установлены следующие компоненты:

– операционная система: Windows 10/11, современный Linux дистрибутив, такой как Linux Mint 22.1, Ubuntu 22.04;

– git: для клонирования репозитория;

– Node.js не ниже версии 22.14.0 и npm не ниже версии 10.9.2;

– python не ниже версии 3.13.2.

Для начала необходимо открыть терминал и клонировать репозиторий командой git clone. Следующим шагом является установление зависимостей. Необходимо создать виртуальное окружение с помощью следующих команд: «python3 -m venv venv», «source venv/bin/activate» для Linux, «venv\Scripts\activate» для Windows. После создания и включения виртуального окружения становится возможным установить необходимые библиотеки с помощью команды «pip install -r requirements.txt». После установки всех зависимостей для бэкенда можно перейти к фронтенду. Для этого в соответствующей директории необходимо выполнить команду «npm install». Запуск всего приложения происходит в два этапа:

– в директории бэкэнда нужно выполнить команду «uvicorn app.main:app –reload»;

– в директории фронтэнда «npm run build».

После выполнения данных действий возможно перейти по адресу localhost:5173 и начать работу с программой.

3.1.2 Работа с интерфейсом

На рисунке 19 представлена страница авторизации в системе с полями ввода логина и пароля. Для использования системы необходимо создать учётную запись.

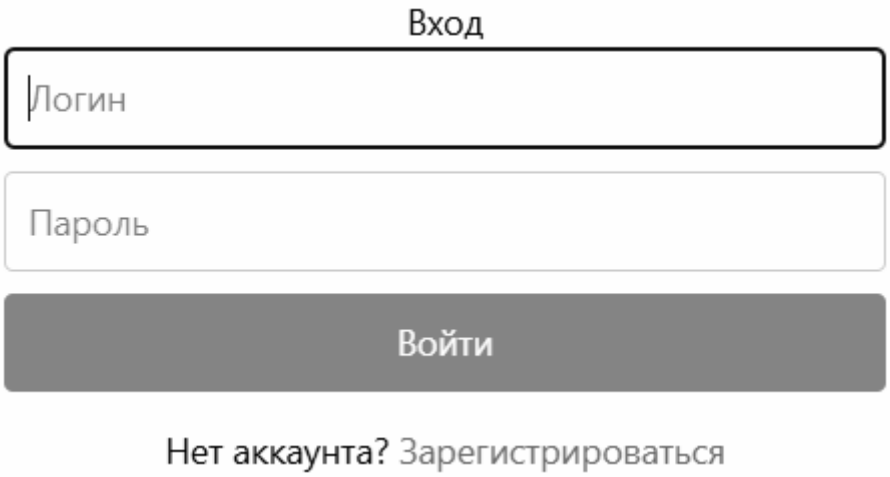


Рисунок 19 – Форма входа

На рисунке 20 представлена форма регистрации нового пользователя. Она включает в себя поля:

– логин, то есть уникальный идентификатор пользователя;

– адрес электронной почты;

– пароль, содержащий не менее 8 символов;

– поле подтверждения пароля.

В случае успешной регистрации система уведомит об этом, и направит на форму входа.

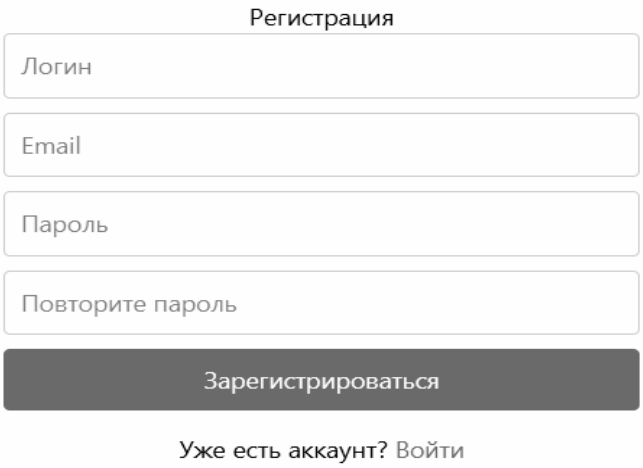


Рисунок 20 – Форма регистрации

После успешного входа отобразится начальная страница веб-приложения представленная на рисунке 21. Страница содержит представленную на русском языке матрицу MITRE ATLAS, кнопку загрузки JSON-файла со сценариями. Ссылку на страницу с уязвимостями. Нажатие на технику вызывает дополнительное окно, с описанием данной техники на русском языке.

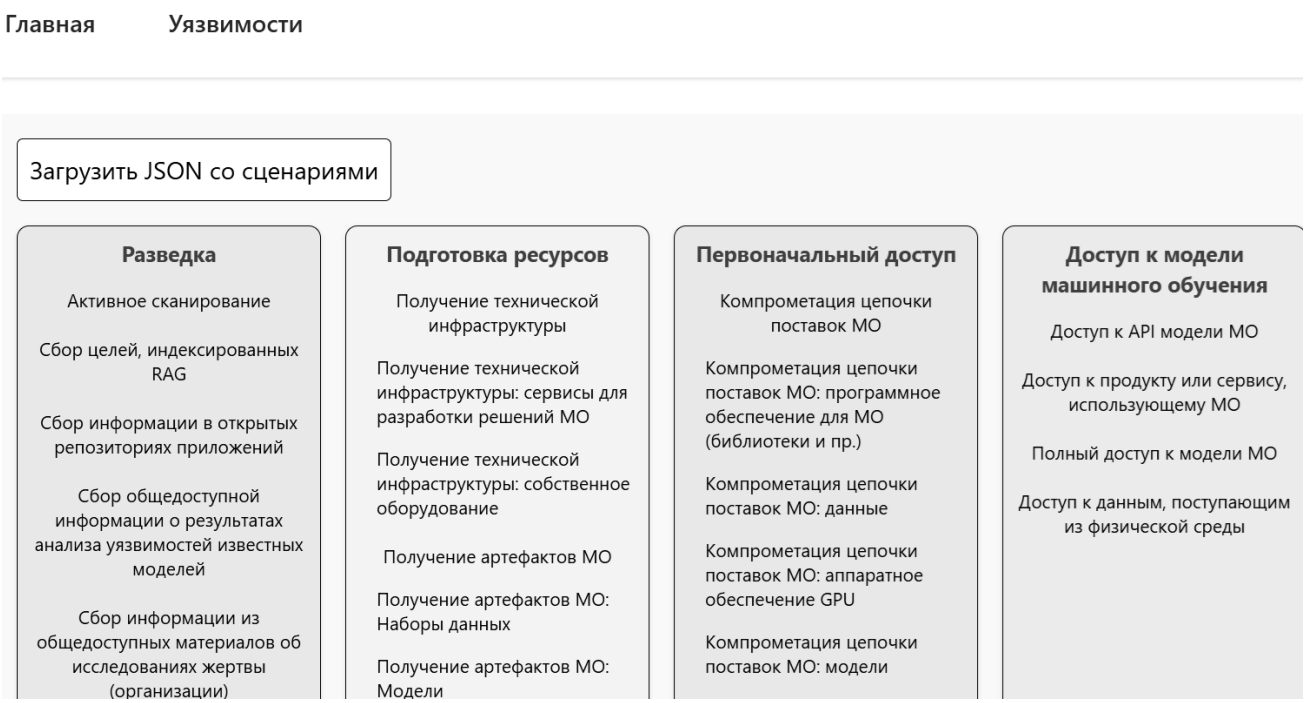


Рисунок 21 – Фрагмент начальной страницы приложения

При нажатии на кнопку «Загрузить JSON со сценариями» откроется диалоговое окно операционной системы Windows или Linux, с возможностью выбрать файл со сценариями, который будет использован для последующей работы.

После подтверждения выбора и загрузки файла внутри веб-интерфейса формируется область предобработки: система распознаёт структуру документа, автоматически извлекает доступные сценарии и предлагает отметить те из них, которые будут участвовать в последующем расчете, что представлено на рисунке 22.

Внизу страницы показывается список используемых техник, полученных из загруженного JSON. Также в этой области размещается кнопка «Построить сценарии», которая служит отправной точкой для запуска построения сценариев, что представлено на рисунке 23.



Рисунок 22 – Фрагмент начальной страницы с загруженным множеством сценариев



Рисунок 23– Фрагмент страницы с техниками и кнопкой построения графа сценариев

При нажатии на кнопку «Построить граф сценариев», в соответствии с алгоритмами, описанными в главе, произойдет построение объединенного графа множества сценариев. Страница с построенным графом представлена на рисунке 24.

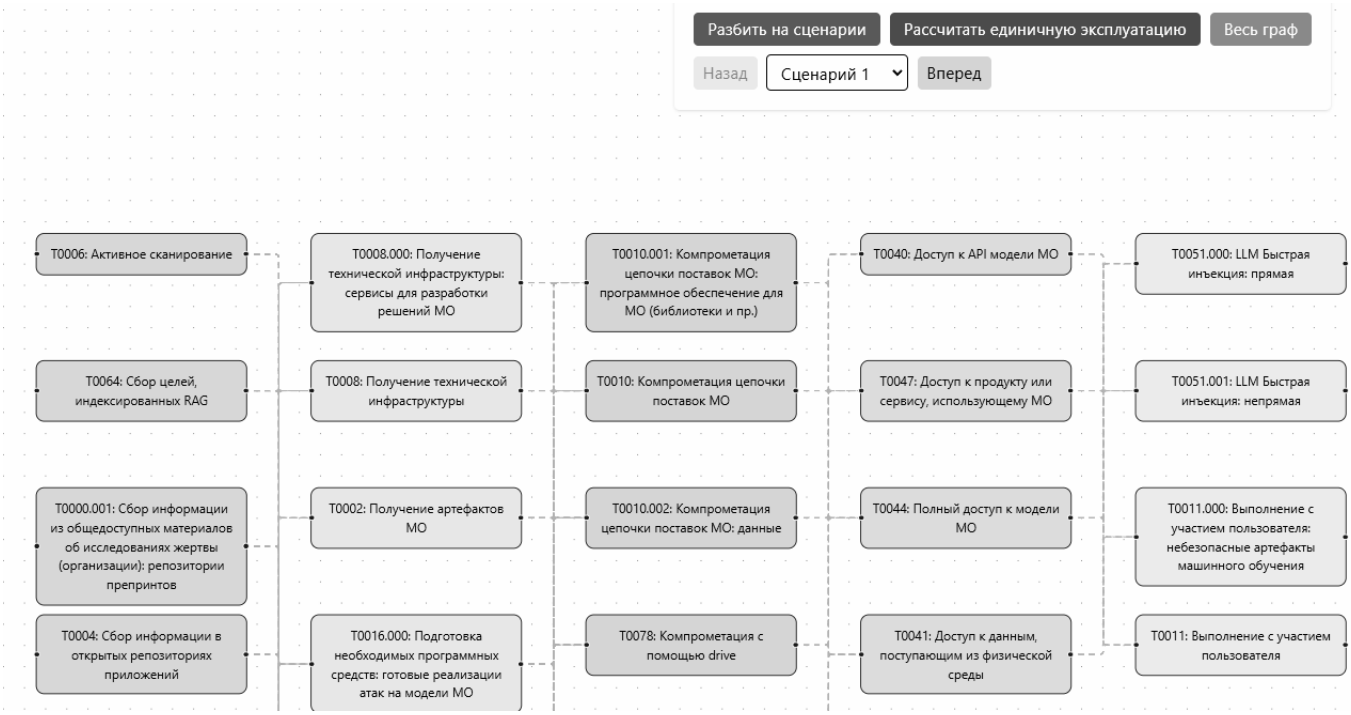


Рисунок 24 – Фрагмент страницы с построенным графом и кнопками управления

Кнопка «Разбить на сценарии» позволяет выделить из общего объединенного графа отдельные сценарии и отобразить их в окне вывода. Кнопка «Рассчитать единичную эксплуатацию» позволяет для выбранного сценария осуществить оценку рисков. Кнопка «Весь граф» возвращает первоначальный вид графа, если было выполнено разбиение. Результат работы программы при нажатии на кнопку «Рассчитать единичную эксплуатацию» представлен на рисунке 25.

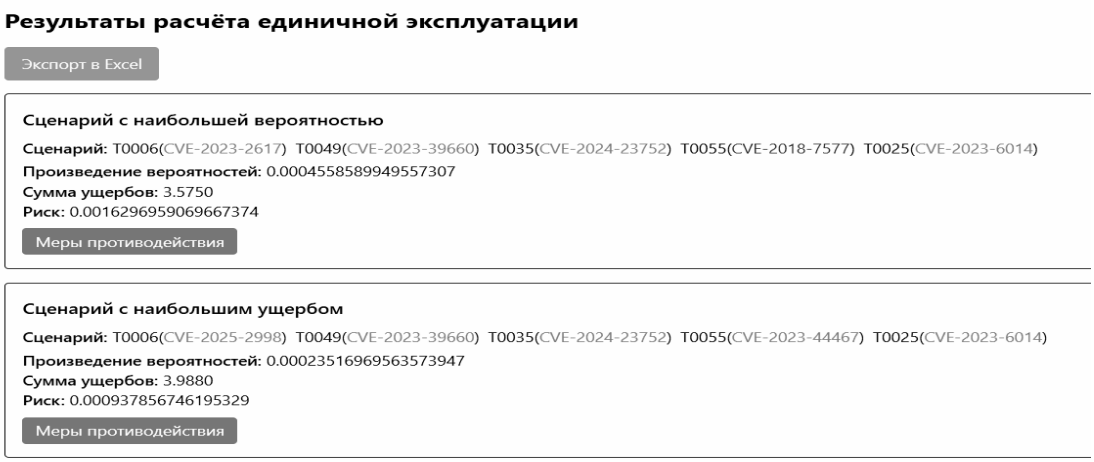


Рисунок 25 – Страница единичной эксплуатации

На рисунке 25 показана страница, на которую перенаправляется пользователь. На данной странице, в соответствии с алгоритмами, описанными во втором разделе, показываются все возможные сценарии. Отдельно выводится сценарии с наибольшей вероятностью и наибольшим ущербом. После показываются все возможные сценарии. Каждая карточка сценария состоит из нескольких ключевых полей:

– сценарий, последовательность техник, и уязвимостей которые они эксплуатируют;

– произведение вероятностей уязвимостей входящих в сценарий;

– сумма ущербов уязвимостей входящий в сценарий;

– итоговое значение риска.

При нажатии на кнопку «Меры противодействия» происходит генерация мер противодействия, которая отображается в отдельном окне.

Для получения сведений обо всех уязвимостях содержащихся в базе данных необходимо перейти на страницу «Уязвимости». Данная страница представлена на рисунке 26.

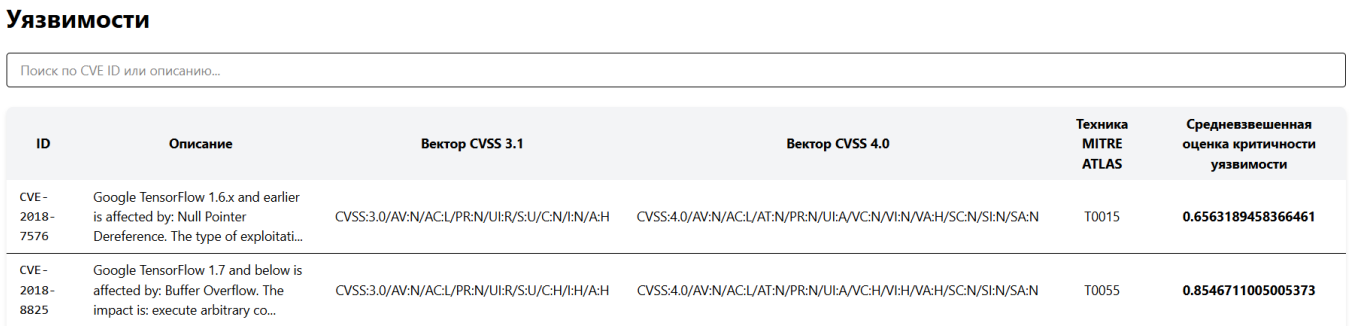


Рисунок 26 – Фрагмент страницы с уязвимостями

На данной странице отображается информация обо всех найденных уязвимостях. Для каждой уязвимости отображаются следующие поля:

– идентификатор уязвимости CVE или BDU;

– описание уязвимости;

– вектор CVSS 3.1;

– вектор CVSS 4.0;

– технику MITRE ATLAS;

– средневзвешенную оценку критичности уязвимости.

При работе со сложной системой неотъемлемой её частью становятся часто возникающие проблемы и способы их устранения. Данный раздел посвящен описанию потенциальных трудностей, которые могут встретить пользователи, и путям их разрешения. Одной из наиболее типичных проблем является невозможность доступа к системе. Это может быть связано с неправильным вводом URL адреса веб-приложения, или временными сбоями в работе сервера. Для предотвращения такой ситуации необходимо перезапустить браузер и проверить корректность адреса веб-приложения. Если проблема не разрешается, необходимо убедится, что серверная часть запущена, и перезапустить ее в таком случае.

Другой категорией сложностей является процесс авторизации и регистрации.

Пользователи могут столкнуться с трудностями при вводе такой информации, как логин пароль, адрес электронной почты, либо попыткой зарегистрироваться с уже существующим именем пользователя. Для успешного прохождения этих этапов важно помнить, что в системе учитывает регистр букв. А также проверка адреса электронной почты на валидность. Пользовательский интерфейс системы включает сообщения об ошибках.

Технические сложности могут проявляться в виде задержек в работе. В частности при оценке риска. Эти проблемы часто обусловлены недостаточной производительностью устройства пользователя. Для их устранения рекомендуется закрыть лишние вкладки в браузере, очистить кэш.

Четкая структура взаимодействия в сочетании с инструкциями по решению типичных неполадок и ошибок обеспечивает простоту и удобство использования системы. Это, в свою очередь, гарантирует высокий уровень доступности и устойчивости в ее функционировании.

3.2 Руководство для разработчиков

3.2.1 Архитектура приложения

Разработка и сопровождение сложных систем машинного обучения предъявляет к разработчикам высокие требования. Недостаточно просто уметь писать код – необходимо понимать внутреннюю организацию системы.

Это руководство предназначено для тех, кто собирается изменять существующий код, расширять функциональность системы или интегрировать её с другими решениями.

В данном разделе описаны ключевые технические аспекты, обеспечивающие корректное и эффективное функционирование системы. Представлена информация о структуре кода, основных компонентах серверной и клиентской сторон, а также советы по их модификации. Особое внимание уделено описанию API-эндпоинтов, благодаря которым осуществляется взаимодействие между пользователями и системой. Дополнительно рассмотрены вопросы интеграции приложения в инфраструктуру, что позволит адаптировать систему под конкретные потребности организаций или проектов.

Главная цель данного руководства – предоставить разработчикам четкий и последовательный подход к работе с кодом системы. Это не только упрощает процесс разработки, но и помогает минимизировать вероятность ошибок, что, в свою очередь, повышает качество итогового продукта.

Архитектура системы построена по модульному принципу. Функции разделены на отдельные компоненты для упрощения поддержки и обеспечения масштабируемости. Основные элементы включают в себя серверную и клиентскую части, каждая из которых выполняет свои задачи и взаимодействует посредством REST API.

Серверная часть реализована на основе функционального, удобного и простого фреймворка FastAPI. Она отвечает за обработку данных, оценку рисков и генерацию мер противодействия. На рисунке 27 представлена структура директорий серверной части приложения.

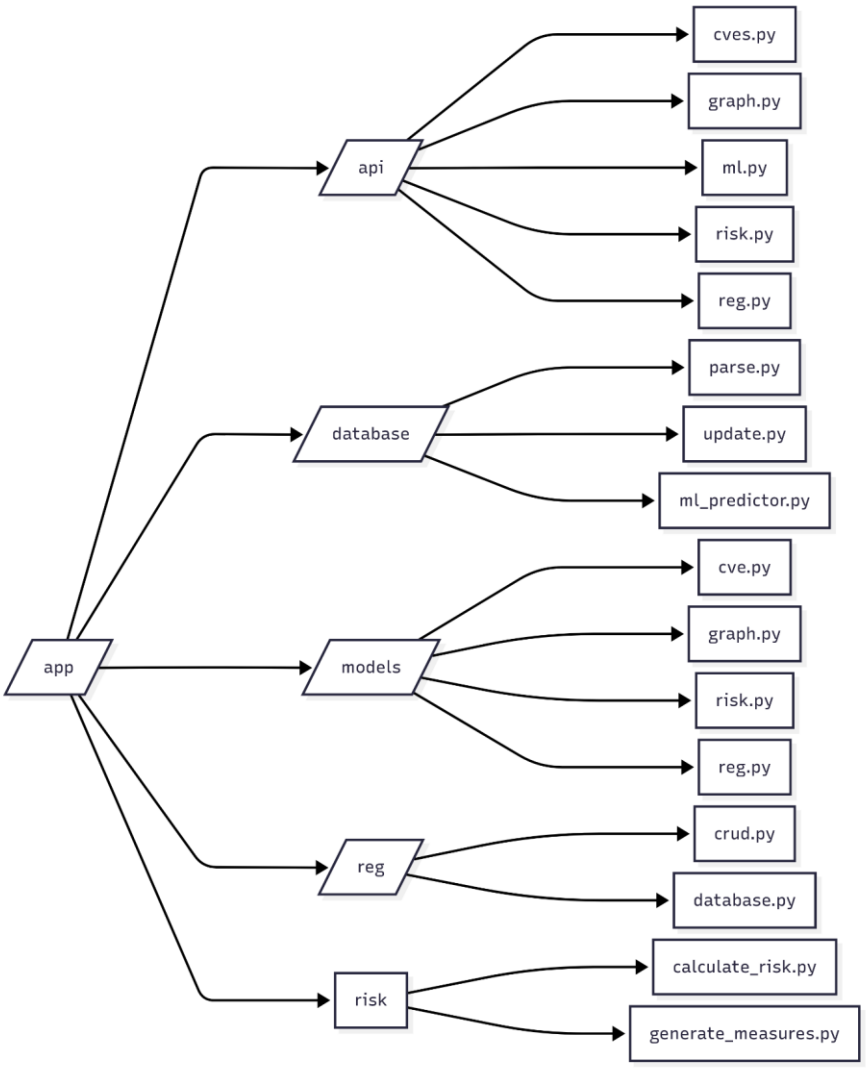


Рисунок 27 – Структура директорий серверной части

В директории «api» содержатся эндпоинты для работе с API, которые разделены на отдельные файлы, в соответствии с функционалом. Директория «database» содержит файлы, ответственные за работу с базой данных. В папке «models» содержатся модели запросов и ответов, для взаимодействия между серверной и клиентской частью. Папка «reg» содержит файлы ответственные за регистрацию и авторизацию. Директория «risk» хранит методы и функции для оценки риска и генерации мер противодействия.

Клиентская часть построена на основе фреймворка React, который позволяет создавать динамический интерфейс с поддержкой взаимодействия в реальном времени. Её основная задача – предоставление удобного и интуитивно понятного интерфейса для работы с системой.

Ключевые элементы клиентской части:

– компоненты страниц. Каждый раздел представлен отдельным компонентом, что облегчает отладку, разработку и дальнейшее тестирование;

– интеграция с API. Библиотека axios используется для отправки запросов, а также обработки токенов авторизации;

– защита маршрутов. Реализована проверка прав доступа, обеспечивая безопасное взаимодействие.

На рисунке 28 представлена структура директорий клиентской части.

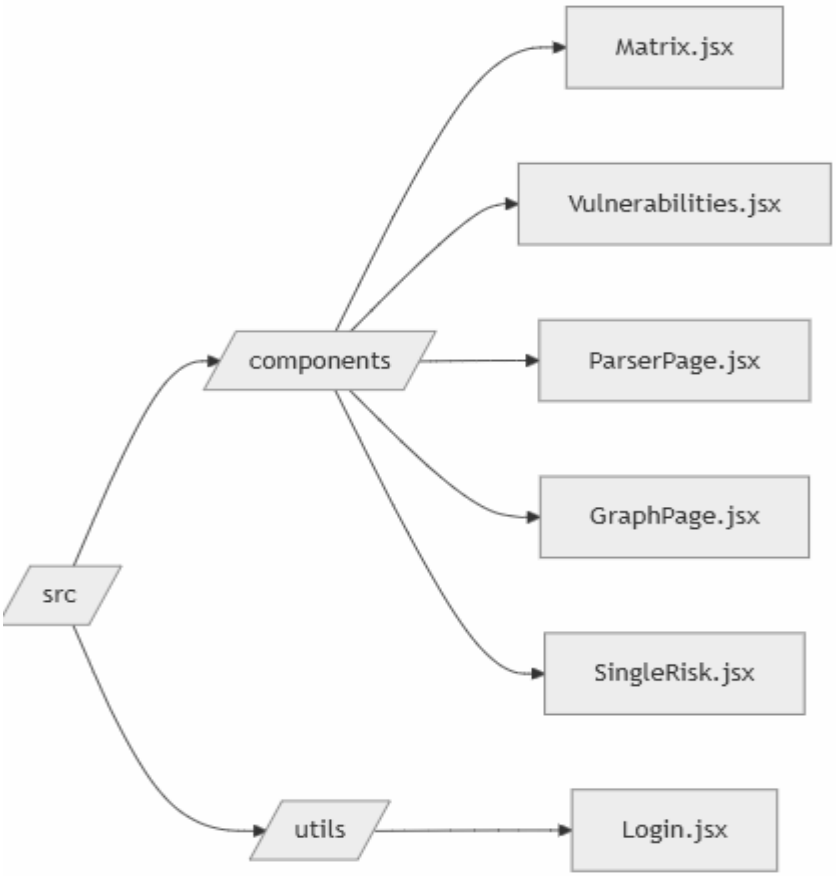


Рисунок 28 – Структура директорий клиентской части

В директории «components» содержатся интерфейсы для работы с приложением. Директория «utils» содержит файлы, ответственные за логику авторизации и обработки токенов.

3.2.2 API и внутренние интерфейсы

Система предоставляет набор REST API, обеспечивающий взаимодействие между клиентом и сервером. Интерфейсы сгруппированы по функционалу:

Агрегирование и вывод сведений об уязвимостях:

– POST /api/cves/nvd – сбор информации об уязвимостях с ресурса NVD;

– POST /api/cves/bdu – сбор информации об уязвимостях с ресурса БДУ ФСТЭК России;

– POST /api/cves/huntr – сбор информации об уязвимостях с ресурса NVD;

– GET /api/cves/{technique_id}/cve – вывод списка уязвимостей связанных с техникой;

– GET /api/cves/list – вывод всех уязвимостей.

Использование средств машинного обучения:

– POST /api/ml/generate_measures – генерация мер противодействия.

Оценка риска:

– POST /api/risk/calculate_path_variants – расчет значения риска для сценария.

Каждый эндпоинт сопровождается документацией, что позволяет быстро понять, для чего он нужен и каким образом его использовать.

Таким образом, руководство предоставляет исчерпывающую информацию для разработчиков, позволяя им эффективно работать с системой, модифицировать ее и интегрировать в различные среды.

3.3 Рекомендации по расширению функционала информационной системы

3.3.1 Новые модули и функции

Развитие информационной системы, можно строить вокруг нескольких ключевых направлений. Прежде всего, важно добавить возможность автоматического запуска модуля агрегирования данных. Такой механизм обновлений значительно сократит время на поддержание базы в актуальном состоянии, и уменьшить вероятность пропустить критический инцидент.

Следующим этапом расширения функционала может стать интеграция с инструментами активного сканирования и обнаружения уязвимостей в инфраструктуре. Налаживание соединения с популярными сканерами даст возможность системе не просто знать об уязвимостях из внешних источников, но и самостоятельно фиксировать их реальное наличие в заданных информационных система. Оценка рисков при этом может основываться на надежных данных о конфигурации и состоянии защищаемых систем, что повысит точность генерации мер противодействия.

Не менее важным шагом может быть внедрение модулей управления процессами и обработки инцидентов. На данный момент система ограничена технической стороной – агрегированием и оценкой риска. Однако в реальных системах жизненный цикл уязвимости включает обнаружение, подтверждение, назначение ответственных, согласование мер и контроль их исполнения. Разработка собственного движка потока задач позволит формировать задачи для ответственных сотрудников, отслеживать статус внедрения мер защиты и вести журнал коммуникаций, что повысит прозрачность процесса и упростит аудит соответствия внутренним политикам и внешним стандартам.

Для поддержки долгосрочного анализа целесообразно расширить аналитическую платформу системы. Можно реализовать накопление исторических данных, построение трендов. Применение методом машинного обучения позволит выявлять закономерности в появлении уязвимостей, прогнозировать их критичность и автоматизировать приоритезацию мер противодействия.

Не следует забывать о расширении интерфейсов взаимодействия: API для интеграции с SIEM-системами и CMDB, поддержка стандартных форматов STIX, а также мобильное приложение для оперативного доступа. Такое комплексное развитие системы позволит повысить ее ценность и упростит принятие решений.

3.3.2 Улучшение пользовательского интерфейса

Улучшение пользовательского интерфейса этой информационной системы должно строиться вокруг принципов простоты восприятия, интуитивности и адаптивности под разные классы пользователей – от технических специалистов до менеджеров рисков. Рабочий стол можно переосмыслить как «живой» центр контроля: вместо сухого списка загруженных сценариев и результатов расчетов рисков можно предоставить визуальные метрики – графики изменения уровня критичности по компонентам, индикаторы количества неподтверждённых уязвимостей и уведомления о предстоящих обновлениях данных.

Во-вторых, навигация по функционалу должна стать более «сквозной»: вместо множества отдельных страниц для загрузки сценариев, просмотра базы стоит объединить схожие задачи в единую последовательность действий. Такой подход уменьшит когнитивную нагрузку на пользователя и сведёт к минимуму необходимость обращаться к справочной документации.

Отдельного внимания заслуживает гибкость интерфейса: механизмы кастомизации представлений позволят адаптировать систему под конкретные роли. Поддержка тёмной и светлой темы оформления, возможность менять размер шрифтов также помогут сделать интерфейс комфортным для длительной работы и разной среды освещения.

Наконец, стоит внедрить продвинутую систему отчетности и экспортирования данных прямо из пользовательского интерфейса: динамические таблицы с возможностью сортировки, группировки и фильтрации сохраняемые в рамках текущей сессии, а также экспорт в PDF с заранее заданными шаблонами. Интерактивные диаграммы с возможностью детализации по конкретным

уязвимостям помогут наглядно демонстрировать ход расследования и принятия мер. Всё это сделает интерфейс не просто красивым, но и по-настоящему полезным инструментом, способным сэкономить время специалистов и повысить эффективность работы с системой.

3.4 Обеспечение информационной безопасности разработанной системы

Обеспечение информационной безопасности системы, агрегирующей данные об уязвимостях и оценивающей риски, должно быть построено по многоуровневому и сквозному принципу, начиная с архитектуры приложения и заканчивая эксплуатационными процессами. На текущем этапе реализована базовая ролевая модель доступа, которая разделяет пользователей на группы с заранее определенными правами: администраторы, пользователи. Несмотря на то, что такая сегрегация функций уже задает каркас безопасности, ее необходимо значительно расширить и углубить для защиты как от внешних, так и от внутренних угроз.

Во-первых, стоит внедрить многофакторную аутентификацию и механизмы единого входа. Сейчас пользователь входит в систему, используя только логин и пароль, что делает его уязвимым к фишинговым атакам и перебору. Подключение второго фактора – одноразового кода из мобильного приложения или аппаратного токена – снизит риск компрометации учетной записи. Интеграция с корпоративным LDAP/Active Directory или OAuth-провайдерами. Это упростит управление учетными записями и позволит централизованно применять политики сложных паролей, блокировок и ротации.

Во-вторых, стоит усилить модель авторизации, перейдя от простой ролевой модели к более детализированной политике на основе атрибутов. Это позволит учитывать не только роль, но и контекст доступа: время суток, местоположение пользователя, характер данных и уровень их критичности.

Важным элементом становится централизованный аудит и логирование всех действий. Необходимо собирать события входа, изменения параметров оценки риска, импорта данных и управления пользователями в неизменяемое хранилище с

поддержкой криптографических подписей. Регулярный анализ логов – как автоматически с помощью SIEM-системы, так и вручную – позволит оперативно выявлять аномалии: множественные неудачные попытки входа, подозрительные операции по массовому экспорту или необоснованное изменение ролей.

Для защиты данных при передаче следует использовать современные криптографические методы: TLS 1.3 для всего сетевого взаимодействия, шифрование базы данных на уровне диска (Transparent Data Encryption) и, при необходимости, шифрование отдельных полей (например, секретных ключей или внутренних токенов). Это гарантирует, что даже в случае физического доступа к серверному оборудованию или перехвата трафика данные останутся недоступными злоумышленникам.

Не менее важно включить в процесс разработки практики безопасного SDLC: регулярный статический и динамический анализ кода, применение принципа «безопасный по умолчанию» и «минимальных привилегий», а также периодические пентесты сторонними специалистами. Такие меры помогут не допустить появления уязвимостей на ранних стадиях, снизив стоимость их исправления и укрепив доверие пользователей.

Наконец, следует разработать и документировать политики реагирования на инциденты: четкий регламент уведомления ответственных, оценку воздействия, коммуникацию с заинтересованными сторонами и пошаговое восстановление работы. Регулярные учения и отработки сценариев утечки или компрометации позволят не только выявить «узкие места» в процессах, но и выстроить слаженное взаимодействие между IT-подразделениями, службой безопасности и руководством компании. В совокупности все перечисленные меры сформируют надежную обеспечат всестороннюю защиту системы на каждом этапе ее жизненного цикла.

ЗАКЛЮЧЕНИЕ

В ходе выполнения исследования была описана и разработана комплексная информационная система для автоматизированного сбора данных об уязвимостях средств машинного обучения, оценки рисков и генерации мер противодействия. Достижение поставленной задачи – повышение защищенности информационных автоматизированных систем использующих средства машинного обучения обеспечено за счет решения следующих ключевых задач:

– анализ существующих информационных ресурсов: проведен обзор открытых баз данных описания атак на средства машинного обучения. Выявлены недостатки существующих подходов, а также пробелы в формализации данных об атаках и рисках;

– разработка алгоритма функционирования системы: сформирована модульная архитектура, объединяющая функции агрегирования сведений об уязвимостях, количественной оценке рисков и автоматизированной выработке мер противодействия;

– программная реализация ключевых модулей: созданы программные компоненты, реализующие автоматический сбор и нормализацию данных об уязвимостях средств машинного обучения, оценку рисков, генерацию мер противодействия.

Новизна полученных результатов заключается в том, что:

– в отличие от аналогов, разработан метод структурированного и автоматизированного представления информации об атаках на средства машинного обучения;

– в отличие от аналогов, предложен алгоритм, объединяющий функции агрегации уязвимостей, оценки рисков и формирования мер противодействия в контексте защиты средств машинного обучения;

– впервые разработан программный комплекс, впервые интегрирующий весь цикл обработки данных об уязвимостях средств машинного обучения – от сбора до противодействия.

Теоретическая значимость результатов, полученных в ходе исследования:

– обеспечивает формальную основу для систематизации и классификации атак на модели машинного обучения с учетом особенностей алгоритмов и архитектур;

– позволяет построить адаптивную систему мониторинга и реагирования, способную автоматически анализировать потоковые данные об уязвимостях в средствах машинного обучения и формировать меры в реальном времени;

– демонстрирует применимость комплексных архитектурных решений для безопасности в области средств машинного обучения.

Практическая значимость результатов, полученных в ходе исследования:

– метод позволяет упростить интеграцию сведений об атаках в системы управления безопасностью средств машинного обучения и ускорить анализ инцидентов;

– алгоритм может быть внедрен в существующие SIEM/SOAR-решения для повышения их эффективности при обнаружении угроз средствам машинного обучения;

– предложенное программное обеспечение обеспечивает автоматизированную защиту средств машинного обучения от специфических угроз за счет оценки рисков и рекомендаций по реагированию.

Таким образом, выполненное исследование не только расширяет теоретические представления об организации защиты средств машинного обучения, но и предлагает готовое техническое решение, способное быть внедренным в практику для обеспечения более высокого уровня безопасности информационных систем.

Перспективы дальнейших исследований связаны с добавлением новых источников уязвимостей, внедрением методов машинного обучения для предсказания новых сценариев и разработкой механизмов защиты информационной системы в режиме реального времени.

СПИСОК ЛИТЕРАТУРЫ

1 Эпидемии в телекоммуникационных сетях / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2017. – 284 с. (Серия «Теория сетевых войн»; вып. 1). – Текст: непосредственный.

2 Атакуемые взвешенные сети / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2017. – 284 с. (Серия «Теория сетевых войн»; вып. 2). – Текст: непосредственный.

3 Социальные сети и деструктивный контент / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2017. – 284 с. (Серия «Теория сетевых войн»; вып. 3). – Текст: непосредственный.

4 Социальные сети и риск-мониторинг / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2019. – 284 с. (Серия «Теория сетевых войн»; вып. 4). – Текст: непосредственный.

5 Социальные сети и психологическая безопасность / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 284 с. (Серия «Теория сетевых войн»; вып. 5). – Текст: непосредственный.

6 Сетео-информационная эпидемиология / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2021. – 284 с. (Серия «Теория сетевых войн»; вып. 6). – Текст: непосредственный.

7 Картография защищаемого киберпространства / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2022. – 372 с. (Серия «Теория сетевых войн»; вып. 7). – Текст: непосредственный.

8 Организационно-правовая защита сетей / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2023. – 228 с. (Серия «Теория сетевых войн»; вып. 8). – Текст: непосредственный.

9 Автоматизированный атлас кибератак / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2025. – 188 с. (Серия «Теория сетевых войн»; вып. 9). – Текст: непосредственный.

10 Мударова Р.М. Противодействие атакам типа инъекция подсказок на большие языковые модели / Мударова Р.М., Намиот Д.Е. // International Journal of Open Information Technologies. –2024. –vol. 12, N.5, –P. 39-46. – Текст: непосредственный

11 AI for Everything URL: https://www.routledge.com/AI-for-Everything/book-series/ – Текст: электронный

12 2024 Информационно-аналитическая справка по результатам мониторинга внедрения решений в сфере искусственного интеллекта в приоритетных отраслях экономики Российской Федерации по итогам 1-го полугодия 2024 года. URL: https://ai.gov.ru/knowledgebase/vnedrenieii/2023_informacionnoanaliticheskaya_spravka_po_rezulytatam_monitoringa_vnedreniya_resheniy_v_sfere_iskusstvennogo_intellekta_v_prioritetnyh_otraslyah_ekonomiki_rossiyskoy_federacii_po_itogam_2023_goda_ncrii/. - Текст: электронный.

13 Котенко И.В. Атаки и методы защиты в системах машинного обучения: анализ современных исследований / И.В. Котенко, И.Б. Саенко, О.С. Лаута, Н.А. Васильев, В.Е. Садовников // Вопросы кибербезопасности. – 2024. – №1(59). –С. 24-37. – Текст: непосредственный

14 Актуальные киберугрозы: II квартал 2024 года. URL: https://www.ptsecurity.com/ru-ru/research/analytics/aktualnye-kiberugrozy-iikvartal-2024-goda/. - Текст: электронный.

15 Актуальные киберугрозы: III квартал 2024 года. URL: https://www.ptsecurity.com/ru-ru/research/analytics/aktualnye-kiberugrozy-iiikvartal-2024-goda/. - Текст: электронный.

16 Group-IB. URL: https://www.groupib.com/products/business-email-protection. - Текст: электронный.

17 MITRE ATLAS URL: https://atlas.mitre.org/ – Текст: электронный

18 MITRE ATT&CK URL: https://attack.mitre.org/ – Текст: электронный

19 MITRE ATLAS Navigator URL: https://mitre-atlas.github.io/atlas-navigator/ – Текст: электронный

20 Сервис моделирования кибератак по матрице MITRE ATT&CK URL: https://mitre.securitycode.ru/ – Текст: электронный

21 Какие техники MITRE ATT&CK выявляют продукты Positive Technologies URL: https://mitre.ptsecurity.com/ru-RU – Текст: электронный

22 Остапенко Г.А. Совершенствование организационно-правового обеспечения информационной безопасности предприятия: формирование риск ландшафта сетевых атак / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. Мирошниченко, 109 А.А. Остапенко, А.Ю. Пекло // Информация и безопасность. 2023. - Т. 26. Вып. 2. - С. 203-210. – Текст: непосредственный

23 Остапенко Г.А. Методики регламентации обеспечения информационной безопасности атакуемых автоматизированных систем / Г.А. Остапенко, А.П. Васильченко // Информация и безопасность.2023. - Т.26.Вып. 4.- С.597-602. – Текст: непосредственный

24 Остапенко Г.А. Формализация знаний и данных кибератак и уязвимостей / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Д.С. Покудин [и другие] // Информация и безопасность. 2024. - Т. 27. Вып. 2. - С. 231-238. – Текст: непосредственный.

25 Остапенко Г.А. Модуль нейросетевой регламентации мер противодействия кибератакам/ Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, А.А. Ноздрюхин [и другие] // Информация и безопасность. 2024. - Т. 27. Вып. 2. - С. 239-246. – Текст: непосредственный.

26 Остапенко Г.А. Модернизация методического обеспечения автоматизированного сервиса агрегации данных и риск-анализа уязвимостей/ Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Н.Н. Корвяков [и другие] // Информация и безопасность. 2024. - Т. 27. Вып. 2. - С. 219-230. – Текст: непосредственный.

27 Банк данных угроз безопасности информации URL: https://bdu.fstec.ru/vul – Текст: электронный

28 National Vulnerability Database URL: https://nvd.nist.gov/vuln – Текст: электронный

29 The world’s first bug bounty platform for AI/ML URL: https://huntr.com/ – Текст: электронный

30 Exploit Prediction Scoring System URL: https://www.first.org/epss/ – Текст: электронный

31 Agentic AI Vulnerability Scoring System URL: https://vineethsai.github.io/aivss/ – Текст: электронный

32 Artificial Intelligence Vulnerability Scoring System URL: https://github.com/OWASP/www-project-artificial-intelligence-vulnerability-scoring-system – Текст: электронный

33 OWASP URL: https://owasp.org/ – Текст: электронный

34 Common Vulnerability Scoring System SIG URL: https://www.first.org/cvss/ – Текст: электронный

35 Common Weakness Enumeration URL: https://cwe.mitre.org/ – Текст: электронный

36 Рындина С. В. Цифровые технологии управления получением, хранением, передачей и обработкой больших данных: SQLite : учеб.-метод. пособие / С. В. Рындина. – Пенза : Изд-во ПГУ, 2023. – 82 с.

37 Асинхронность как основной подход к разработке высоконагруженных приложений URL: https://habr.com/ru/articles/733154/ – Текст: электронный

38 WebDriver URL: https://www.w3.org/TR/webdriver2/ – Текст: электронный

39 Official Common Platform Enumeration (CPE) Dictionary URL: https://nvd.nist.gov/products/cpe –Текст: электронный

40 Остапенко А.А. Оценка критичности уязвимостей с использованием данных множества риск калькуляторов / А.А Остапенко // Информация и безопасность. 2024. Т.27.Вып. 4. С. 543-552 – Текст: непосредственный

41 Остапенко А.А. Актуальные задачи и некоторые их решения в части риск калькуляции успешности реализации единичных кибератак / А.А. Остапенко // Информация и безопасность. 2024. Т. 27. № 4. С. 543-552.

42 CMAD_V1 URL: https://github.com/0wct1/cmad – Текст: электронный

43 Вьюгин В.В. Математические основы теории машинного обучения и прогнозирования / В.В. Вьюгин. – Москва: МЦМНО, 2013. – 390 с. – ISBN 978-5-4439-0111-4. – Текст: непосредственный

44 Хасти Т. Элементы статистического обучения / Т. Хасти, Р. Тибширани, Дж. Х. Фридман – Нью-Йорк: Springer. 2009. 763 с.: ISBN 978-0-387-84857-0. – Текст: непосредственный

45 Фридман Дж. Х. Усиление стохастического градиента URL: https://web.archive.org/web/20140801033113/http://statweb.stanford.edu/~jhf/ftp/stobst.pdf. –Текст: электронный

46 Изучение представлений признаков с помощью метода К-средних URL: https://web.archive.org/web/20150621073741/http://www.cs.stanford.edu/~acoates/papers/coatesng_nntot2012.pdf – Текст: электронный

47 Эстер М. Алгоритм на основе плотности для обнаружения кластеров в больших пространственных базах данных с шумом / М. Эстер, Х.-П. Кригель, Й. Сандер, С. Сюй // Труды Второй международной конференции по обнаружению знаний и интеллектуальному анализу данных –AAAI Press, 1996. С. 226-231. –ISBN 1-57735-004-9. – Текст: непосредственный

48 Метрики качества работы моделей машинного обучения: как их понимать и использовать. URL: https://webiomed.ru/blog/metrikikachestva-raboty-modelei/. - Текст: электронный.

49 Макмахан Б. Знакомство с PyTorch: глубокое обучение при обработке естественного языка / Б. Макмахан, Д.Рао. – Питер, 2020. – 256 с. – ISBN 978-5-4461-1241-8. – Текст: непосредственный

50 Давыдов А.В. Сравнение различных языков программирования, применяемых в машинном обучении / А.В. Давыдов, А.К. Жусупова, О.С. Салыкова // Вестник науки. 2023. Т.3. №2 (59). С. 155-165 – Текст: непосредственный

51 Топ-5 языков для машинного обучения URL: https://www.itweek.ru/ai/article/detail.php?ID=207784 – Текст: электронный

52 «8 Reasons Why Python is Good for AI and ML» URL: https://djangostars.com/blog/why-python-is-good-for-artificial-intelligence-and-machine-learning/ – Текст: электронный

53 Остапенко Г.А. Модуль нейросетевой регламентации мер противодействия кибератакам / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, А.А. Ноздрюхин, Д.С. Покудин, Н.Н. Корвяков // Информация и безопасность. 2024. Т. 27. Вып. 2. С. 239-246. – Текст: непосредственный.

54 Остапенко Г.А. Организационно-правовая защита от сетевых атак: методики формирования частных политик, регламентов и инструкций обеспечения безопасности организации (часть I) / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. Мирошниченко, А.А. Остапенко, А.С. Кривошеин // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 329-340. – Текст: непосредственный.

55 Остапенко Г.А. Организационно-правовая защита от сетевых атак: методики формирования частных политик, регламентов и инструкций обеспечения безопасности организации (часть II) / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. 152 Мирошниченко, А.А. Остапенко, А.Г. Краснобородкин // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 329-340. – Текст: непосредственный.

ПРИЛОЖЕНИЕ А

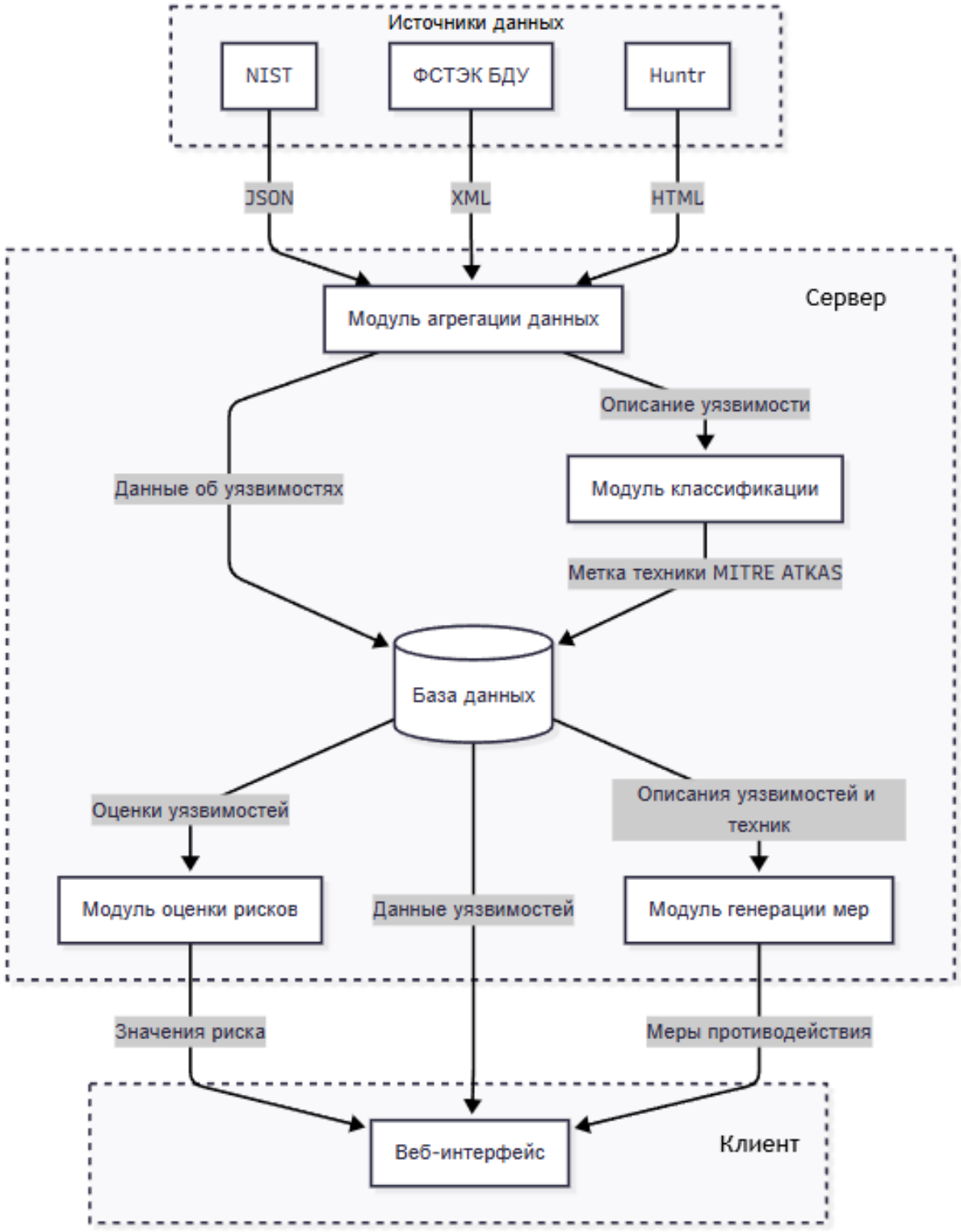


Рисунок А1 – Схема взаимодействия модулей

ПРИЛОЖЕНИЕ Б

Фрагменты ключевых алгоритмов

1. Получение данных

1.1 NVD NIST

```
def fetch_vulnerabilities(keywordSearch: str = None, cpeName: str = None, cve_list: list[str] = None,max_pages: int = 10) -> list:
vulnerabilities = []
    results_per_page = 2000
    for page in tqdm(range(0, max_pages)):
        params = {
            "startIndex": page * results_per_page,
            "resultsPerPage": results_per_page
        }
        response = requests.get(API_URL, params=params)
        if response.status_code != 200:
            break
        data = response.json()
        for item in data.get("vulnerabilities", []):
          vulnerabilities.append({
                "id": vuln_id,
                "cwe": cwe,
                "description": description,
                "cvss31": cvss_vector,
                "cvss31c": cvss_score,
                "cvss4": cvss4_vector,
                "cvss4c": cvss4_score})
    return vulnerabilities
```

1.2 БДУ ФСТЭК России

```
def download_excel(url: str, output_path: str):
    response = requests.get(url, stream=True, verify=False)
    response.raise_for_status()
      with open(output_path, 'wb') as f:
        for chunk in response.iter_content(chunk_size=8192):
            if chunk:
                f.write(chunk)
if __name__ == "__main__":
    excel_url = BDU_FILE_URL
    save_path = "vullist.xlsx"
    try:
        download_excel(excel_url, save_path)
        print(f"Файл сохранён: {save_path}")
    except requests.HTTPError as e:
        print(f"Ошибка при скачивании: {e}")
def parse_bdu_file(file_path: str, sheet_name: Optional[str] = None) -> List[Dict]:
    base_dir = os.path.dirname(os.path.abspath(__file__))
    if not os.path.isabs(file_path):
        file_full_path = os.path.join(base_dir, file_path)
    else:
        file_full_path = file_path
    if not os.path.exists(file_full_path):
        raise FileNotFoundError(f"Файл не найден: {file_full_path}")
    sheet_to_read = sheet_name if sheet_name is not None else 0
    df = pd.read_excel(file_full_path, sheet_name=sheet_to_read, dtype=str)
    vulnerabilities: List[Dict] = []
    for _, row in df.iterrows():
        vid = row.iloc[0] if pd.notna(row.iloc[0]) else None
```

```
        desc = row.iloc[1] if len(row) > 1 and pd.notna(row.iloc[1]) else None
        cvss3_vec = row.iloc[11] if len(row) > 11 and pd.notna(row.iloc[11]) else None
        cvss4_vec = "N/A"
        severity_text = row.iloc[12] if len(row) > 12 and pd.notna(row.iloc[12]) else None
        cvss3_score = _extract_score(severity_text)
        cvss4_score = 0.0
        cwe = row.iloc[-1] if pd.notna(row.iloc[-1]) else None
        vulnerabilities.append({
            "id": vid,
            "description": desc,
            "cwe": cwe,
            "cvss31": cvss3_vec,
            "cvss31c": cvss3_score,
            "cvss4": cvss4_vec,
            "cvss4c": cvss4_score
        })
    return vulnerabilities
```

1.3 EPSS

```
def get_epss_score(cve_id: str) -> dict:
    try:
        response = requests.get(EPSS_API_URL, params={"cve": cve_id})
        response.raise_for_status()
        data = response.json()
        result = data.get("data", [{}])[0]
        epss = (result.get("epss", 0))
        epss = epss.replace(',', '.')
        return {
            "epss": float(epss),
            "percentile": float(result.get("percentile", 0))
```

```
        }
    except Exception as e:
        print(f"[EPSS] Error for {cve_id}: {e}")
        return {
            "epss": 0.0,
            "percentile": 0.0
        }
```

1.4 Запуск агрегирования

```
def run_pipeline(keywordSearch: str = None, cpeName: str = None,cve_list: list[str] = None,
                 max_pages: int = 10):
    if cve_list and isinstance(cve_list, str):
        cve_list = [cve_list]

    raw_vulns = fetch_vulnerabilities(
        keywordSearch=keywordSearch,
        cpeName=cpeName,
        cve_list=cve_list,
        max_pages=max_pages
    )
    vulns: list[dict] = []
    for item in raw_vulns:
        if isinstance(item, dict):
            vulns.append(item)
        elif isinstance(item, list):
            for sub in item:
                if isinstance(sub, dict):
                    vulns.append(sub)
```

```
results = []
for vuln in tqdm(vulns):
    cve_id = vuln.get("id")
    if not cve_id:
        continue
    epss = get_epss_score(cve_id)
    aivss = aivss_score(vuln["cvss31"])
    technique = predict_technique(vuln["description"], vuln["cwe"])
    final_description = vuln["description"]
    if(vuln["cvss4"] == "N/A"):
        vuln["cvss4"] = convert_cvss3_to_cvss4(vuln["cvss31"])
        vuln["cvss4c"] = get_cvss40_score(vuln["cvss4"])

    cia = get_cia(vuln["cvss31"])
    final_score = combine_scores(
        float(vuln["cvss31c"]),
        float(vuln["cvss4c"]),
        float(epss["epss"]),
        float(aivss)
    )
    vuln_data = {
        "cve_id": vuln["id"],
        "cwe": vuln["cwe"],
        "description": final_description,
        "technique": technique,
        "cvss31": vuln["cvss31"],
        "cvss31c": vuln["cvss31c"],
        "cvss4": vuln["cvss4"],
        "cvss4c":vuln["cvss4c"],
        "epss": epss["epss"],
```

```
            "aivss_score": aivss,
            "final_score_average": final_score[0],
            "final_score_sg": final_score[1],
            "cia":cia
            }
        save_to_db(vuln_data)
        results.append(vuln_data)
    return results
```

2. Предсказание техники MITRE ATLAS

```
def predict_technique(description: str, cwe_id: str) -> str:
    inf_tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
    inf_model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)
    inf_model.eval()
    text = f"CWE-{cwe_id} {description}"
    inputs = inf_tokenizer(
        text,
        truncation=True,
        padding='max_length',
        max_length=512,
        return_tensors='pt'
    )
    with torch.no_grad():
        outputs = inf_model(**inputs)
        logits = outputs.logits
        pred_id = torch.argmax(logits, dim=1).item()
    return id2label[pred_id]
```

3. Расчет критичности уязвимости

```
def combine_scores(cvss31_score: float, cvss4_score: float, epss_score: float, aivss_score: float) -> float:
    try:
        cvss31_score = (cvss31_score - 0) / (10 - 0)
        cvss4_score = (cvss4_score - 0) / (10 - 0)
        epss_score = (epss_score - 0) / (1 - 0)
        aivss_score = (aivss_score  - 0) / (10 - 0)
        average_score = (cvss31_score + cvss4_score + epss_score + aivss_score ) / 4
        std = math.sqrt(((cvss31_score - average_score) ** 2 + (cvss4_score - average_score) ** 2 + (epss_score - average_score) ** 2 + (aivss_score - average_score) ** 2) / 3)
        sg_cvss31 = cvss31_score / std
        sg_cvss4 = cvss4_score / std
        sg_epss = epss_score / std
        sg_aivss = aivss_score / std
        sg_norm_cvss31 = sg_cvss31 / (sg_cvss31 + sg_cvss4 + sg_epss + sg_aivss)
        sg_norm_cvss4 = sg_cvss4 / (sg_cvss31 + sg_cvss4 + sg_epss + sg_aivss)
        sg_norm_epss = sg_epss / (sg_cvss31 + sg_cvss4 + sg_epss + sg_aivss)
        sg_norm_aivss = sg_aivss / (sg_cvss31 + sg_cvss4 + sg_epss + sg_aivss)
        sg_score_cvss31 = cvss31_score * sg_norm_cvss31
        sg_score_cvss4 = cvss4_score * sg_norm_cvss4
        sg_score_epss = epss_score * sg_norm_epss
        sg_score_aivss = aivss_score * sg_norm_aivss
    final_score_sg = sg_score_cvss31 + sg_score_cvss4 + sg_score_epss + sg_score_aivss
    except:
        final_score_sg = 0
    return final_score_sg
```

4. Расчет значения ущерба

```
def get_cia(cvss3_vector: str) -> str:
    try:
        confidentialy_impact = float(metric_to_number(cvss3_vector[-1]))
    except:
        confidentialy_impact = 0
    try:
        integrity_impact = float(metric_to_number(cvss3_vector[-5]))
    except:
        integrity_impact = 0
    try:
        availability_impact = float(metric_to_number(cvss3_vector[-9]))
    except:
        availability_impact = 0
    impact = 1 - (1 - confidentialy_impact) * (1 - integrity_impact) * (1 - availability_impact)
    return impact
```

5. Лучевой поиск

```
def calculate_path_variants(
    req: PathVariantsRequest,
    db: Session = Depends(get_db)
) -> PathVariantsResponse:
     technique_vulns: List[List[dict]] = []
    total_combinations = 1
    for tech_id in req.techniques:
        vulns = db.query(cves).filter(cves.technique == tech_id).all()
        if not vulns:
            continue
       weighted_scores = []
```

```
    for v in vulns_sorted:
        e = 1 if v.have_exploit else 0
        b = b_values.get(v.exploit_code_maturity, 1)
        weighted_scores.append(v.final_score_sg * (1 + b * e))
    total_sev = sum(weighted_scores)
    if total_sev <= 0:
        raise HTTPException(status_code=400)
    items = []
    for v, w in zip(vulns_sorted, weighted_scores):
        items.append({
            "tech": tech_id,
            "cve": v.cve_id,
             "severity": w / total_sev,
            "impact": v.cia,
            "have_exploit": v.have_exploit,
        })
    technique_vulns.append(items)
beam: List[Tuple[List[dict], float, float]] = [([], 1.0, 0.0)]
for items in technique_vulns:
    new_beam: List[Tuple[List[dict], float, float]] = []
    for path_items, sev_prod, imp_sum in beam:
        for item in items:
            new_path = path_items + [item]
            new_sev = sev_prod * item['severity']
            new_imp = imp_sum + item['impact']
            new_beam.append((new_path, new_sev, new_imp))
    new_beam.sort(key=lambda t: t[1] * t[2], reverse=True)
    beam = new_beam[:req.limit]
  variants: List[VariantResponse] = []
for path_items, sev_prod, imp_sum in beam:
```

```
        variants.append(VariantResponse(
            path=[PathItem(**item) for item in path_items],
            risk=sev_prod * imp_sum,
            severity_product=sev_prod,
            impact_sum=imp_sum
        ))
    max_sev_variant = max(variants, key=lambda v: v.severity_product)
    max_imp_variant = max(variants, key=lambda v: v.impact_sum)
    return PathVariantsResponse(
        variants=variants,
        max_severity_variant=max_sev_variant,
        max_impact_variant=max_imp_variant,
        total_possible=total_combinations
    )
```

6. Генерация мер противодействия

```
def preprocess_batch(batch):
    inputs = [f"Описание уязвимости: {d}\nMITRE Atlas: {m}" for d, m in zip(batch[desc_col], batch[mitre_input_col])]
    labels = []
    for i in range(len(inputs)):
        row = batch
        text = (
            "Базовые меры:\n"
            f"1) Организационно-правовые: {row[label_cols['bo']][i]}\n"
            f"2) Технические: {row[label_cols['bt']][i]}\n"
            "Компенсирующие меры:\n"
            f"1) Организационно-правовые: {row[label_cols['co']][i]}\n"
            f"2) Технические: {row[label_cols['ct']][i]}\n"
            f"Меры реагирования: {row[label_cols['resp']][i]}\n"
```

```
            f"Меры ликвидации последствий: {row[label_cols['rem']][i]}\n"
            f"Превентивные меры: {row[label_cols['prev']][i]}"
        )
        labels.append(text)
    model_inputs = tokenizer(inputs, max_length=512, truncation=True)
    tokenized_labels = tokenizer(labels, max_length=512, truncation=True)
    model_inputs['labels'] = tokenized_labels['input_ids']
    return model_inputs
tokenizer = T5Tokenizer.from_pretrained(OUTPUT_DIR)
model = T5ForConditionalGeneration.from_pretrained(OUTPUT_DIR).to(device)
def generate_mitigation(description, mitre_list, max_length=256, num_beams=4):
    mitre_str = mitre_list
    inp = tokenizer(f"Описание уязвимости: {description}\nMITRE Atlas: {mitre_str}",
return_tensors='pt')
    input_ids = inp.input_ids.to(device)
    attention_mask = inp.attention_mask.to(device)
    outputs = model.generate(
        input_ids,
        attention_mask=attention_mask,
        max_length=max_length,
        num_beams=num_beams,
        no_repeat_ngram_size=3,
        repetition_penalty=2.0,
        length_penalty=1.2,
        early_stopping=True
    )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```